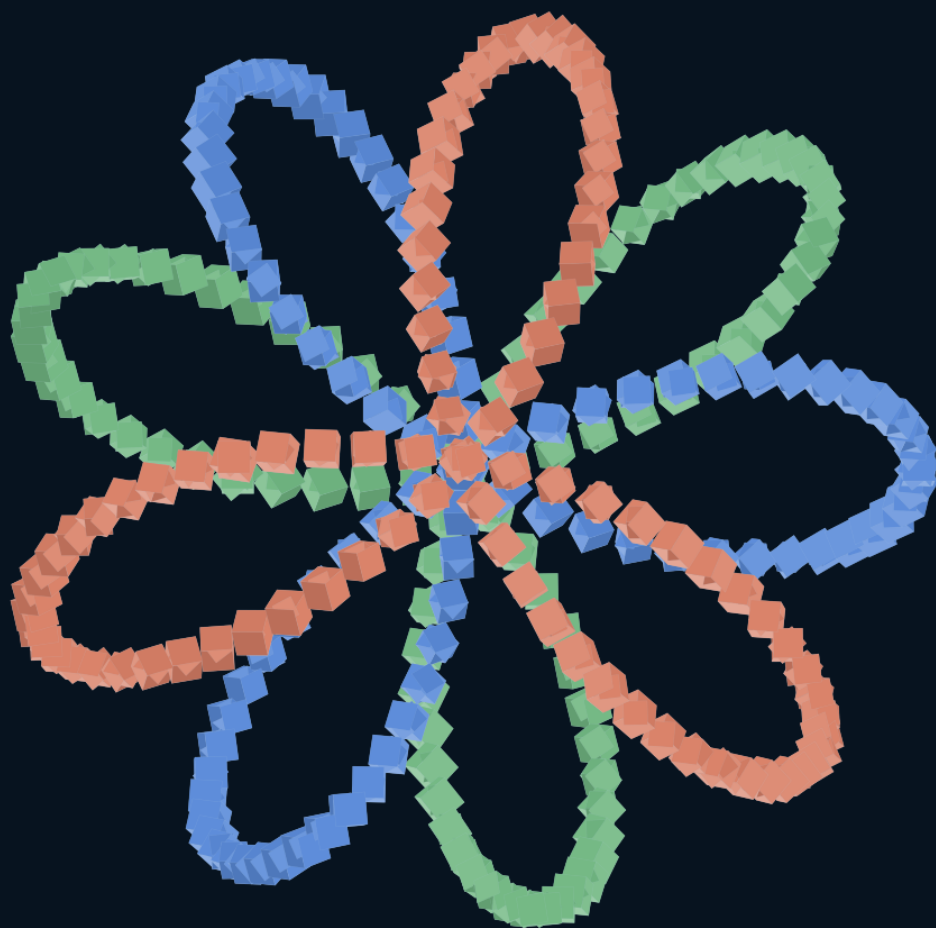




EDIÇÃO 0.6

SPATIALSEED

Livro, manual da linguagem e atlas procedural

Arquitetura · categorias · AST · cores · cidade · obras reproduzíveis

Rogério Duarte

24 de julho de 2026 · commit b4043c6

Sumário

O livro alterna fundamentos, manual operacional, atlas procedural e evidência. Os arquivos de código estão anexados ao PDF.

PARTE I — POR QUE UM MUNDO PRECISA DE CONTRATOS 4

SpatialSeed em uma frase	5
O problema do “mesmo mundo”	7
Invariantes: o que não pode depender da interface	9
Categorias como disciplina de projeto	10

PARTE II — ARQUITETURA, SUBSISTEMAS E PAINÉIS 12

Três opções arquiteturais	13
As cinco camadas	14
Comando, reduzir, delta e histórico	16
Protótipos, instâncias e recursos	17
O mapa dos painéis	18

PARTE III — MANUAL DA LINGUAGEM E DA INTERFACE 20

Início rápido	21
Modelo lexical	22
Catálogo de comandos	23
Seleção	24
Criação e transformações	25
Pivôs, gizmo e snapping	26
Duplicação, matriz delta e repeat	27
Gramática matemática	28
Semântica indexada	30
AST canônica e independência da sintaxe	32
Cor, material e textura	33
Projetos, apresentação, testes e diagnósticos	34

PARTE IV — GERAÇÃO PROCEDURAL: DA REGRA À CIDADE 36

O modelo mental	37
Exemplo 1: Onda Integrada	38
Exemplo 2: Hélice Ascendente	40
Exemplo 3: Roseta Tricromática	42
Exemplo 4: Cidade Policêntrica	45

Obra avançada: Trindade Orbital	49
---	----

PARTE V — PERFORMANCE, TESTES E HONESTIDADE EXPERIMENTAL 52

O que foi medido	53
O que os novos exemplos medem — e o que não medem	55
Como medir corretamente	56

PARTE VI — POSSIBILIDADES E ROTEIRO 57

Possibilidades imediatas	58
Possibilidades distribuídas	59
Motivações para continuar	60

PARTE VII — TEMPO, EXPERIMENTOS E COLABORAÇÃO 61

Do 0022 ao 0028e	62
Laboratórios declarativos	63
Tempo sem corromper a autoria	64
Colaboração verificável com LLMs	65

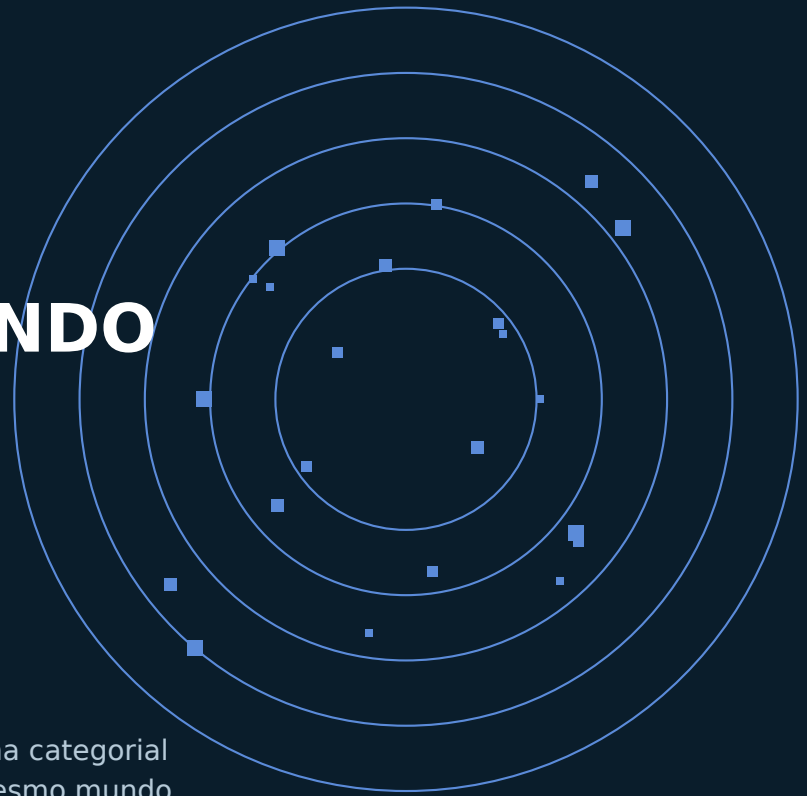
APÊNDICES 67

Referência rápida de comandos	69
Referência matemática	71
Programas executáveis	72
Evidência, fontes e limites	75
Checklist de reprodução	76
Nota editorial da versão 0.6	77

FUNDAMENTOS

POR QUE UM MUNDO PRECISA DE CONTRATOS

Identidade, autoridade, memória e a disciplina categorial
que mantém interfaces diferentes sobre o mesmo mundo.



Um ambiente espacial pode começar como uma cena, mas só se torna um mundo quando adquire identidade, memória e regras para mudar sem deixar de ser reconhecível.

1

SpatialSeed em uma frase

SpatialSeed é um ambiente espacial modular para **criar, editar, simular, publicar e habitar mundos digitais**. A frase é deliberadamente mais ampla que “editor 3D”. Um editor pode encerrar sua responsabilidade quando salva uma cena; um mundo precisa continuar existindo quando a interface muda, quando outro viewer o projeta, quando uma região aceita ou rejeita uma proposta e quando a autoria se distribui entre pessoas, programas e máquinas.

O nome contém a tese do projeto. Uma semente não é a miniatura geométrica de tudo que virá. É uma estrutura pequena o bastante para ser transportada e precisa o bastante para regenerar relações. No SpatialSeed, essa estrutura é formada por identificadores estáveis, schemas, comandos versionados, ASTs, recursos endereçáveis, snapshots e deltas. A imagem é uma projeção dessa semente; não é sua única verdade.

A pergunta de projeto não é apenas “como desenhar isto?”, mas “qual é a menor descrição que permite que isto volte a existir, seja inspecionado e continue sendo o mesmo mundo?”

Esta edição está ancorada no `main` promovido em 20 de julho de 2026, commit [b4043c6](#), build [20260720-0028e](#). Ela preserva os exemplos procedurais validados no 0021d e incorpora a evolução real dos ciclos 0022 a 0028e. Para não confundir ambição com entrega, cada afirmação usa um destes estatutos:

Estatuto	Significado no livro	Exemplo
Implementado	Existe no código do commit de referência	hierarquia, PWA, procedimentos, experimentos, animação efêmera
Testado	Possui evidência automática ou roteiro executado	suíte integral 337/337 validada no 0028d; escala uniforme validada no 0028e
Decisão	Deve orientar mudanças futuras	comandos canônicos, preview separado de commit
Requisito	Comportamento solicitado ainda não concluído	origens individuais na manipulação múltipla
Opção	Alternativa preservada para decisão posterior	persistência de clips ou receita temporal
Horizonte	Arquitetura-alvo, sem alegação de entrega	autoridade regional remota, colaboração multiusuário

Essa distinção impede que a ambição filosófica seja confundida com uma lista de funcionalidades prontas. O projeto pode pensar adiante sem falsificar seu presente.

O que o usuário vê hoje

No navegador, há uma cena WebGL, ferramentas de seleção e transformação, hierarquia, pivôs, histórico local, arquivos de projeto, Inspector coletivo, criação geométrica, laboratório de experimentos, painel de animação, painéis simultâneos, modo somente-cena e um console capaz de executar comandos, expressões, procedimentos e programas. Botões e atalhos convergem para ações semânticas; ações, Inspector, console e programas convergem para comandos e serviços públicos.

O que a arquitetura protege

Por baixo da interface, o projeto procura preservar seis propriedades:

- **identidade:** objetos e regiões não se reduzem à posição de um pixel;
- **autoridade:** cada mudança persistente tem um lugar legítimo de aceitação;
- **composição:** comandos pequenos podem formar programas maiores;
- **memória:** estados, deltas e ASTs permitem reconstrução e auditoria;
- **substituibilidade:** viewer, painel ou idioma podem mudar sem redefinir o mundo;
- **limites:** seleção, hover, câmera e preview não fingem ser estado canônico.

2

O problema do “mesmo mundo”

Imagine três interfaces. A primeira arrasta um cubo com um gizmo. A segunda executa `move 1 0 0`. A terceira recebe um delta pela rede. Se cada uma mantiver uma implementação privada da transformação, há três mundos apenas visualmente parecidos. Pequenas diferenças de pivô, ordem de rotação, unidade ou arredondamento acumulam divergência.

SpatialSeed procura uma condição mais forte: as interfaces devem produzir o mesmo comando semântico ou comandos declaradamente distintos. A convergência não acontece no nível do botão; acontece no contrato compartilhado.



Figura 1 — A ação gráfica e o programa textual convergem para um comando canônico. As setas tracejadas representam a exigência de equivalência observável.

O quadrado da figura “comuta” quando seguir o caminho gesto → comando → reduzir conduz ao mesmo estado que uma tradução direta semanticamente válida. A expressão vem da teoria das categorias, mas aqui funciona como teste de engenharia: rotas diferentes entre os mesmos pontos devem concordar sobre o que foi alterado.

Identidade não é aparência

Um objeto pode mudar de material, posição ou viewer e conservar seu identificador. Duas instâncias podem compartilhar geometria e material, mas manter identidades diferentes. Uma seleção pode destacar temporariamente uma instância sem alterar sua cor lógica. Essas diferenças parecem pequenas até que undo, sincronização, autoria e publicação dependam delas.

Autoridade não é visibilidade

O renderer vê o mundo, porém não deve decidir seu estado. O viewer conhece câmera, hover e seleção, mas a região não precisa conhecer o ponteiro do usuário. O sandbox pode guardar uma proposta não aceita. A autoridade é a camada autorizada a transformar uma intenção em estado canônico — não a camada que desenha a consequência mais chamativa.

Memória não é um monte de snapshots

Salvar o mundo inteiro a cada gesto é possível em protótipos, mas não resolve por si só a semântica. Uma memória útil registra também **qual operação** ocorreu, sobre **qual versão**, com **quais argumentos**, e qual delta resultou. É isso que permite revisar, reaplicar, rejeitar, rebasear e comparar.

3

Invariantes: o que não pode depender da interface

Uma arquitetura modular só é real se algumas afirmações continuarem verdadeiras quando módulos são trocados. No SpatialSeed, os invariantes centrais são:

- 1 Todo estado compartilhado é tratável como snapshot imutável.
- 2 Toda mudança persistente atravessa um comando ou envelope equivalente.
- 3 Toda mudança aceita produz um delta versionado ou informação suficiente para derivá-lo.
- 4 Renderer e viewer são projeções; não são autoridade do mundo.
- 5 A região não conhece estado transitório do editor, como hover ou arraste intermediário.
- 6 Réplicas conhecem a versão-base de que partiram.
- 7 Recursos compartilhados permanecem compartilhados até que uma edição exija distinção.
- 8 Uma interface nova deve conseguir reutilizar contratos existentes sem copiar o núcleo.

Esses invariantes não proíbem experiências locais. Ao contrário: um sandbox pode ser radical precisamente porque sua diferença em relação ao estado aceito é explícita.

4

Categorias como disciplina de projeto

Teoria das categorias não aparece aqui como ornamento terminológico nem como afirmação de que o código atual implementa uma biblioteca categorial. Ela oferece uma linguagem para discutir **objetos, transformações, composição e invariantes entre representações**.

Estados e comandos

Considere uma categoria conceitual W_{mundo} :

- os objetos de W_{mundo} são estados válidos do mundo;
- um morfismo $c : W_0 \rightarrow W_1$ é um comando aceito que leva o estado W_0 ao estado W_1 ;
- a composição $c_2 \circ c_1$ é uma sequência válida de comandos;
- a identidade id_W é a operação que preserva W .

O reducer pode ser lido como uma ação parcial $R(W, c) = W'$. Ela é parcial porque nem todo comando é válido em todo estado: selecionar um identificador inexistente ou aplicar uma escala inválida deve falhar em vez de inventar um resultado.

Sintaxe e semântica

Uma frase do console pertence a uma categoria sintática S_{fonte} ; a AST canônica pertence a uma categoria intermediária A_{ast} ; transformações avaliadas pertencem a uma categoria operacional T_{eval} . O pipeline pode ser descrito por traduções:

$$S_{\text{fonte}} \rightarrow A_{\text{ast}} \rightarrow T_{\text{eval}} \rightarrow W_{\text{mundo}}$$

O objetivo é que a troca de palavras-chave ou idioma altere a primeira seta, não o significado persistido em A_{ast} . A AST 0021d registra versão, hash, linguagem de origem, modo e IDs semânticos. Assim, a frase é uma porta de entrada; a AST é a semente regenerável.



Figura 2 — Pipeline da linguagem: fonte, tokens, AST canônica, avaliação indexada, transformações, sandbox e viewer.

Viewers como funtores

Um viewer pode ser pensado como um funtor $V : W_{\text{mundo}} \rightarrow G_{\text{cena}}$, em que G_{cena} contém cenas e alterações gráficas. Essa imagem conceitual impõe duas perguntas práticas:

- o viewer preserva identidade e composição relevantes?
- uma mudança de viewer altera apenas a projeção ou redefine silenciosamente o mundo?

Dois viewers não precisam produzir pixels idênticos. Um pode usar WebGL e outro outline textual. A equivalência desejada é semântica: ambos devem reconhecer os mesmos objetos, versões e comandos observáveis. Construir um segundo viewer é, por isso, um teste de arquitetura, não uma duplicação cosmética.

Naturalidade entre interfaces

Quando um painel visual e o console traduzem intenções para o mesmo registro, podemos exigir uma forma de naturalidade: mudar o estado e depois projetar deve concordar com projetar e aplicar a atualização correspondente. Na prática, isso significa evitar caminhos ocultos em que um widget altera diretamente um mesh sem atualizar sandbox, histórico e projeto.

Procedural como família indexada

Uma obra procedural pode ser descrita por uma função

$$\gamma : I \rightarrow \text{Aff}(3)$$

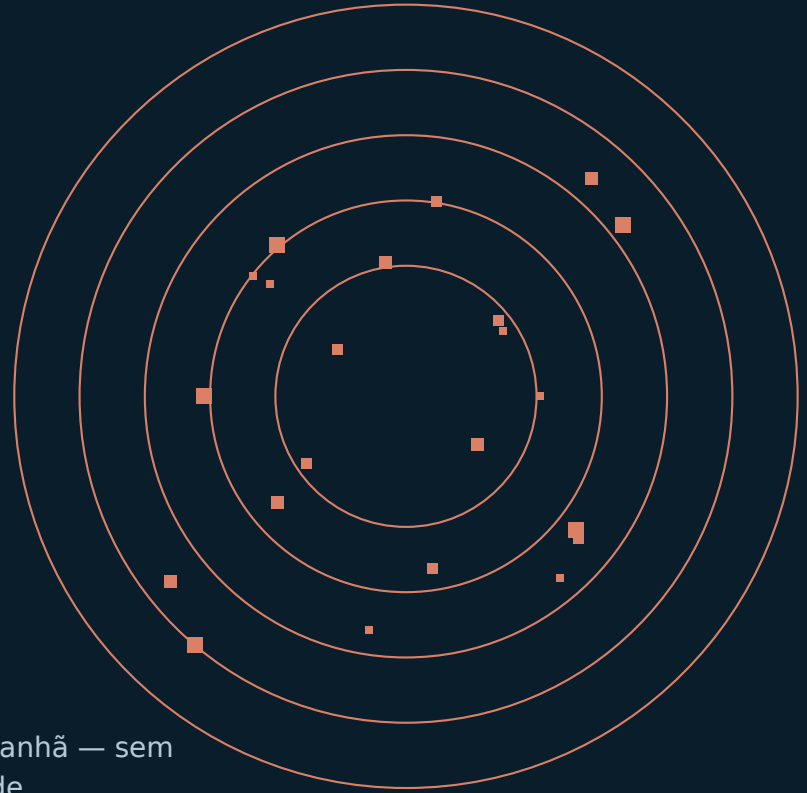
que associa cada índice $i \in I$ a uma transformação afim. Dada uma semente 0_0 , a família é

$$0_i = \gamma(i) \cdot 0_0.$$

O índice é mais que um contador: carrega i , u , count , tempo e o estado transformado. Alterar a função altera a família inteira sem editar objeto por objeto. Essa é a motivação matemática do capítulo de geração procedural.

ARQUITETURA

ARQUITETURA, SUBSISTEMAS E PAINÉIS



Camadas locais hoje, regiões distribuídas amanhã — sem confundir viewer, editor, sandbox e autoridade.

A arquitetura atual é local e estratificada; a arquitetura-alvo admite regiões autoritativas distribuídas. O livro mantém as duas escalas visíveis.

5

Três opções arquiteturais

Não existe uma arquitetura universalmente correta. Existem custos que se tornam aceitáveis ou perigosos conforme o objetivo.

Opção	Vantagem	Limite	Quando escolher
Editor monolítico	prototipagem rápida, poucas fronteiras	UI, estado e renderer se contaminam; difícil substituir	prova visual descartável
Runtime local estratificado	contratos testáveis, undo, console e viewer convergem	ainda não prova coordenação remota	estado atual do SpatialSeed 0028e
Mundo regional distribuído	autoridade explícita, réplicas, deltas, publicação federada	protocolos, conflitos e observabilidade mais complexos	evolução para mundos compartilhados

O branch de referência está deliberadamente no meio. Ele não simula uma rede distribuída apenas para parecer avançado. Em vez disso, constrói fronteiras locais que poderão atravessar processos e redes sem serem inventadas do zero.

Opção A — monólito orientado à interface

O botão chama diretamente um método do renderer; o mesh muda; um arquivo serializa o que conseguir encontrar. É a rota mais curta para uma demonstração. Também é a rota em que undo, console, automação e sincronização viram exceções.

Opção B — runtime local estratificado

O viewer traduz gesto em comando; o comando opera sobre seleção e sandbox; o reducer produz novo estado e mudanças; o renderer projeta. O custo são mais tipos, testes e diagnósticos. O benefício é que caminhos distintos convergem.

Opção C — autoridade regional distribuída

O sandbox torna-se réplica especulativa. Uma proposta carrega versão-base; a autoridade regional valida e aceita, rejeita ou exige rebase; deltas retornam às réplicas. Viewer e editor continuam locais. Essa opção é o horizonte do projeto, não uma alegação de que o GitHub Pages já hospeda um servidor autoritativo.

6

As cinco camadas

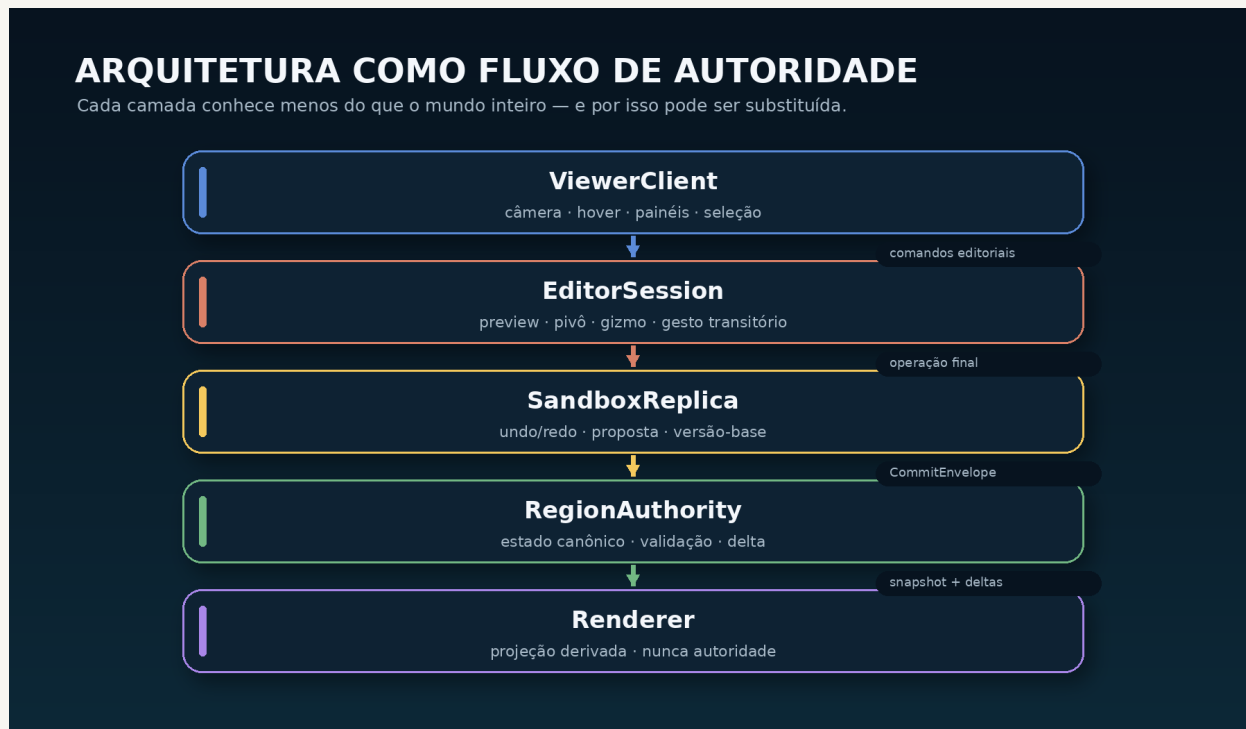


Figura 3 — Fluxo de autoridade entre viewer, sessão editorial, sandbox, região e renderer.

ViewerClient

Conhece câmera, apontador, hover, seleção visual, atalhos e painéis. Pode desaparecer sem destruir o mundo. O modo somente-cena confirma essa separação: a cena continua quando o HTML é ocultado.

EditorSession

Conhece ferramentas, gizmo, pivôs, snapping e previews. Durante um arraste, dezenas de atualizações visuais podem ocorrer; somente a operação confirmada precisa atravessar a fronteira persistente. Isso reduz ruído e evita que a autoridade conheça gestos incompletos.

SandboxReplica

Mantém estado local, versão-base, dirty flag, undo/redo e proposta. É o espaço onde experimentar não equivale a publicar. No 0028e a autoridade continua local, mas o vocabulário e as fronteiras já diferenciam réplica, edição, região, preview temporal e projeção.

RegionAuthority

É responsável pelo estado canônico da região: objetos persistentes, versão, validação e deltas. Na arquitetura distribuída, recebe `CommitEnvelope`; na implementação local, o mesmo limite é exercitado pelo reducer e pelo objeto Region.

Renderer

Transforma estado em projeção gráfica, administra recursos e seleção visual. Pode usar instancing, caches e helpers, desde que não altere o estado autoritativo por conta própria.

7

Comando, reducer, delta e histórico

O núcleo de uma ação persistente pode ser lido assim:

```
intenção → comando canônico → validação → reducer → novo estado + mudanças
                                     ↓
                               histórico · renderer · eventos
```

Um comando não precisa saber desenhar. Um reducer não precisa saber onde o botão está. Um renderer não precisa interpretar a intenção original. Essa restrição reduz acoplamento e torna os testes menores.

Registro canônico

O `CommandRegistry` associa um ID a um handler e metadados. Botões e console chamam os mesmos IDs, como `selection.rotate`, `pivot.relative` e `project.save`. O registro permite listar capacidades e medir a fachada de runtime sem duplicar lógica.

Reducer regional

O reducer recebe estado e comando, retorna estado e lista de mudanças. Comandos desconhecidos ou inválidos não deveriam produzir mutação invisível. As mudanças alimentam classificação incremental, diagnósticos e renderer.

Undo e redo

Undo/redo pertencem ao sandbox local. Eles não são equivalentes a apagar história global. Num mundo distribuído, desfazer pode significar propor um comando inverso, restaurar uma versão ou abandonar uma proposta local; são políticas diferentes e devem permanecer nomeadas.

Repeat

Depois de uma duplicação com transformação constante, o sistema registra uma matriz delta $\Delta = M_{\text{final}} \cdot M_{\text{inicial}}^{-1}$. `repeat` reaplica essa matriz à fronteira atual. Programas paramétricos não habilitam `repeat` no 0021d, porque continuar corretamente exigiria preservar índice e programa canônico.

Protótipos, instâncias e recursos

O mundo lógico e o custo gráfico não precisam crescer na mesma proporção. Objetos idênticos podem compartilhar geometria e material; cada instância guarda identidade, transformação e atributos próprios.

Protótipo e instância

O protótipo descreve recursos compartilháveis. A instância referencia esse conjunto e acrescenta o que a torna particular. Editar somente uma instância pode exigir copy-on-write: um novo protótipo derivado é criado, e as demais continuam no original.

Lotes de instâncias

`InstancedMesh` permite que muitos objetos lógicos usem poucas chamadas gráficas. O `InstanceBatchManager` mantém a relação entre identificador lógico e índice de lote. Um índice reutilizado deve receber transformação e cor novas, nunca resíduos do ocupante anterior.

Cor por instância

Uma cor própria usa `instanceColor`; ela não muda `batchKey`, não cria material nem geometria e acrescenta nominalmente 12 bytes por capacidade de instância. A seleção usa cor temporária; ao deselegionar, o renderer restaura a cor lógica.

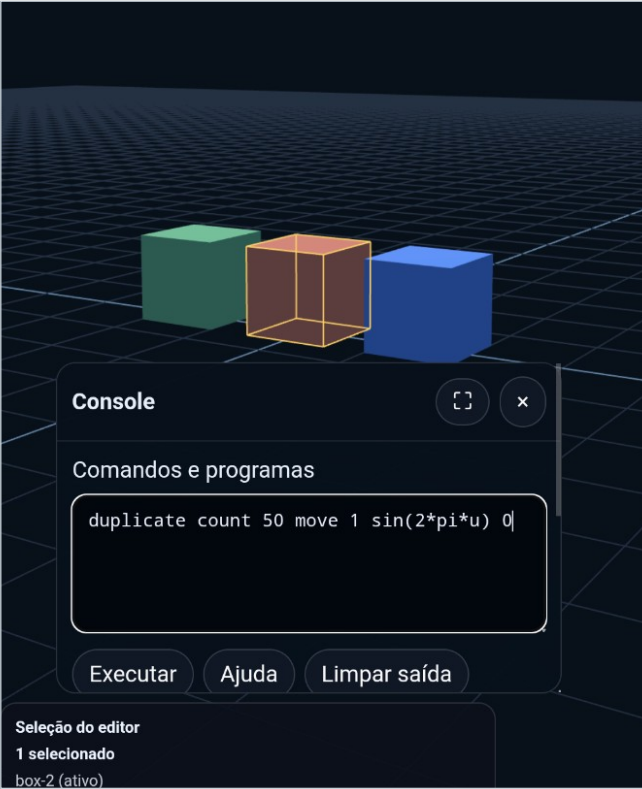


Figura 4 — Material compartilhado, cor da instância e destaque transitório têm responsabilidades distintas.

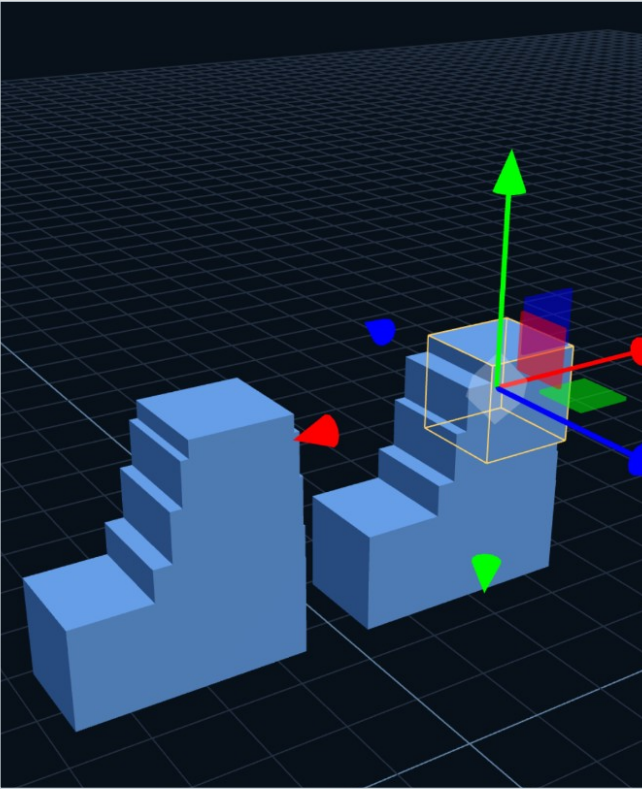
O mapa dos painéis

O SpatialSeed 0028e distribui a interface em superfícies especializadas e configuráveis. Nem todos os painéis são autoridades; muitos são instrumentos de projeção, inspeção ou tradução.

Superfície	Responsabilidade	Estado que toca
Toolbar	escolher ferramenta, espaço e operação de seleção	EditorState e comandos
Painel de seleção	resumo de membros, ativo e pivô	projeção de EditorState
Console	programa textual, inspeção, testes e benchmarks	CommandRegistry e queries
Inspector	transformação, dimensões, material, cor e textura	patch de objeto no sandbox
Transformação	gizmo, eixos, snapping e grid lock	configuração do renderer/editor
Estrutura	estado regional e objetos	projeção do sandbox/região
Diagnóstico	execução e entrada	dados de observabilidade
Desenvolvedor	runtime, recursos, incremental e renderer	queries; não autoridade
Projeto	salvar, abrir e reiniciar	ProjectService
Cena/tela cheia	apresentação sem HTML ou em fullscreen	apenas viewer



Console paramétrico sobre a cena; o teclado foi removido do enquadramento editorial.



Gizmo tridimensional com objeto ativo e eixos de transformação.

Toolbar e seleção

Os modos navegar, selecionar, mover, girar e escalar mudam a interpretação do gesto. Operações `replace`, `add`, `remove` e `toggle` definem como o hit-test altera a seleção. Seleção por área produz vários membros; o objeto ativo permanece distinguível.

Inspector

Edita um objeto por vez. Além de nome, posição, rotação, escala e dimensões, expõe cor-base compartilhada, cor própria da instância e textura. “Aplicar ao sandbox” confirma o patch; selecionar não deve modificar o objeto.

Transformação

Controla tamanho e eixos do gizmo, snapping de translação/rotação/escala e bloqueio de grid. A orientação mundo/local pertence ao contexto editorial. O gizmo projeta uma transformação; o comando confirmado é a autoridade persistente.

Estrutura, diagnóstico e desenvolvedor

Estrutura responde “o que existe”; Diagnóstico responde “como a entrada e a execução foram interpretadas”; Desenvolvedor responde “quais camadas, recursos e métricas estão ativos”. Separar essas perguntas evita um painel onisciente e intrusivo.

Projeto e publicação

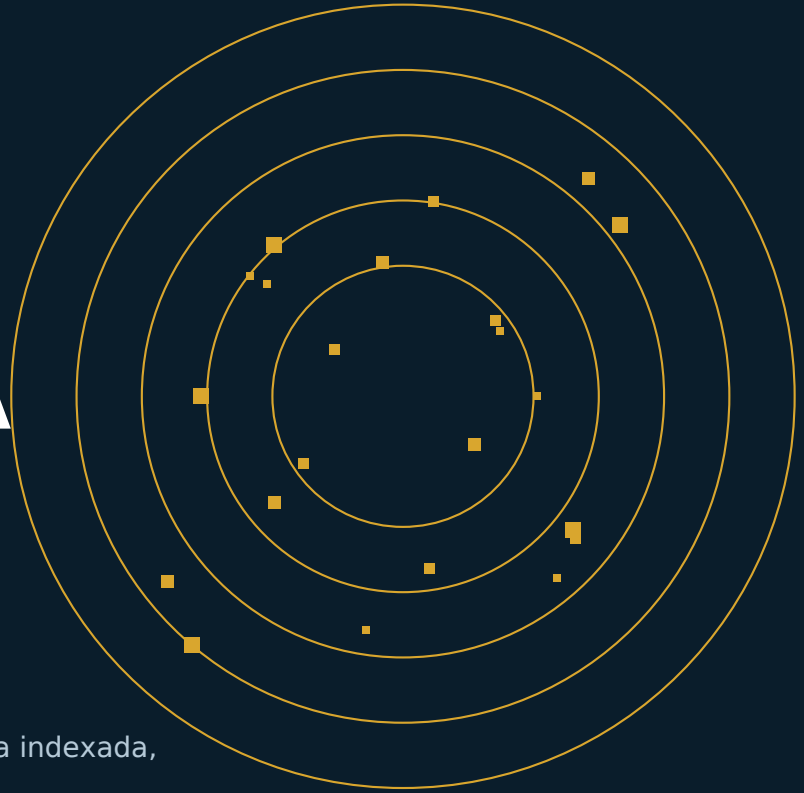
Salvar serializa cena, editor, aparência e configuração relevante. Abrir valida e restaura. O GitHub Pages publica a aplicação estática, não o estado privado do usuário. A URL pública usada nesta edição é:

[Abrir o SpatialSeed público no GitHub Pages](#)

MANUAL

MANUAL DA LINGUAGEM E DA INTERFACE

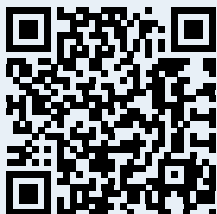
Comandos, gramática matemática, semântica indexada,
AST, cor, projetos, diagnósticos e testes.



A linguagem é pequena, mas não é uma coleção arbitrária de macros. Ela possui regras lexicais, contexto matemático, operações afins e uma AST canônica versionada.

10

Início rápido



Experiência pública

Teste a edição pública no navegador. O livro está ancorado no commit b4043c6; uma publicação futura pode apresentar diferenças.

[abrir SpatialSeed no GitHub Pages](#)

- 1 Abra a URL pública.
- 2 Se houver trabalho importante na cena, use **Salvar** antes de continuar.
- 3 Recarregue a página para recuperar as sementes `box-1`, `box-2` e `box-3`.
- 4 Abra **Console**.
- 5 Cole um programa e pressione **Executar** uma vez.
- 6 Use **Cena** para ocultar os painéis e ajuste a câmera por órbita e zoom.
- 7 Use `select clear` antes da captura para remover o destaque de seleção.

O console aceita um comando por linha ou comandos separados por ponto e vírgula:

```
select only box-2
duplicate count 12 move 1 0 0 rotate 0 15 0
select clear
```

Estado inicial

ID	Cor-base	Posição inicial	Uso didático nesta edição
box-1	azul #5b8bd9	[-3, 1, 0]	hélice e distrito azul
box-2	coral #d98067	[0, 1, 0]	onda e distrito coral
box-3	verde #72b883	[3, 1, 0]	eixos verdes e terceira fase

Novo cria um projeto vazio. Programas que selecionam IDs iniciais devem começar após recarregar, não após **Novo**.

11

Modelo lexical

Separadores

Quebra de linha e ; encerram um comando. Linhas vazias são ignoradas. Não há comentário textual na gramática 0021d; remova explicações antes de colar.

Tokens

Fora de aspas, espaços separam tokens. Expressões matemáticas com espaços devem permanecer entre aspas simples ou duplas. O tokenizer remove somente as aspas externas.

```
duplicate count 24 move "3 * cos(i * pi / 12)" "i * 0.2" "3 * sin(i * pi / 12)"
```

Números e validação

Comandos simples como `move` e `scale` exigem números imediatos. Expressões são aceitas dentro de operações de `duplicate count`. Quantidades devem ser inteiras positivas; o limite atual é 100.000 cópias por chamada. Escalas simples devem ser positivas, embora a linguagem matemática possa avaliar escala assinada em ensaios específicos.

Erros

Cada linha é executada separadamente e recebe resultado `ok` ou erro. Uma linha posterior pode executar mesmo que uma anterior falhe. Para programas destrutivos, valide primeiro com poucas cópias.

12

Catálogo de comandos

Família	Comando	Efeito
Ajuda	<code>help, commands</code>	sintaxe e registro de capacidades
Inspeção	<code>inspect ...</code> , <code>list objects</code>	lê seleção, entrada, sandbox, região ou objetos
Seleção	<code>select only/add/remove/toggle</code> , <code>select clear</code> , <code>clear</code>	altera membros do editor
Criação	<code>create box [x y z]</code>	cria e seleciona uma caixa
Transformação	<code>position</code> , <code>move</code> , <code>rotate</code> , <code>scale</code>	transforma seleção
Pivô	<code>pivot</code> <code>median/bounds/active/absolute/relative</code>	escolhe ou define centro
Procedural	<code>duplicate</code> , <code>duplicate count</code> , <code>repeat</code> , <code>delete</code>	gera, repete ou remove
Histórico	<code>undo</code> , <code>redo</code>	histórico do sandbox local
Ferramentas	<code>vertices</code> , <code>snap</code> , <code>gizmo</code>	configuração e diagnóstico editorial
Testes	<code>test ...</code> , <code>runtime test ...</code>	executa suítes
Performance	<code>benchmark ...</code> , <code>runtime benchmark api</code>	cenas e fachada de runtime
Recursos	<code>runtime resources</code>	lotes, materiais, geometria e memória

Inspeção

```
inspect selection
inspect selected
inspect selected all
inspect input
inspect editor
inspect sandbox
inspect region
inspect objects
list objects
```

`inspect selected` retorna o objeto ativo; `inspect selected all` retorna todos os selecionados. Inspeção não deve alterar estado.

13

Seleção

Seleção é estado editorial, não componente persistente do objeto. Cada membro registra `kind`, `regionId` e `objectId`; um membro pode ser ativo.

```
select only box-1
select add box-2 box-3
select remove box-2
select toggle box-3
select clear
```

- `only` substitui a seleção;
- `add` inclui sem remover os atuais;
- `remove` exclui membros nomeados;
- `toggle` inverte a presença;
- `clear` esvazia.

O comando legado `select box-1 box-2` continua selecionando os IDs informados. Prefira a forma explícita em programas reproduzíveis.

Seleção e identidade

O console rejeita IDs inexistentes. Isso é importante: selecionar “o objeto aproximadamente nesta posição” e selecionar `box-1` são operações semanticamente diferentes. A primeira depende de projeção e hit-test; a segunda depende de identidade.

14

Criação e transformações

Criar

```
create box  
create box 3 1 -2
```

Uma caixa criada recebe UUID, nome incremental, tamanho padrão e cor-base. O objeto recém-criado torna-se a seleção atual.

Position e move

```
position 0 4 0  
move 1 0 -2
```

`position` coloca o pivô efetivo da seleção numa coordenada mundial absoluta. `move` adiciona um delta a cada objeto selecionado. Para um único objeto com pivô padrão, `position` coincide com mover o centro até o alvo; para grupos e pivôs personalizados, a distinção importa.

Rotate

```
rotate 0 30 0
```

Os três argumentos são graus em ordem XYZ. A rotação opera em torno do pivô efetivo. No programa paramétrico, a operação `rotate` é incremental por índice no modo `indexed`.

Scale

```
scale 1 2 1
```

O comando simples multiplica a escala atual. Na semântica paramétrica `indexed`, `scale` é interpretado como **fator da escala da semente no índice**, evitando crescimento exponencial acidental. Essa diferença é central e será detalhada adiante.

15

Pivôs, gizmo e snapping

Políticas de pivô

```
pivot median
pivot bounds
pivot active
```

- **median**: média das origens selecionadas;
- **bounds**: centro da caixa delimitadora conjunta;
- **active**: origem do objeto ativo.

Pivô personalizado

```
pivot absolute 0 0 0
pivot relative 1 0 0
```

O pivô absoluto permanece em coordenadas mundiais. O relativo é armazenado em relação à origem do ativo e acompanha a mudança de contexto.

Pivô dentro de duplicate

```
duplicate count 12 move 1 0 0 pivot active rotate 0 15 0
duplicate count 12 pivot absolute 0 0 0 rotate 0 30 0
```

A posição da cláusula participa da ordem do programa. A duplicação registra o pivô solicitado e o resolvido, tornando o resultado auditável.

Snapping e vértices

```
snap move 0.5
snap rotate 15
snap scale 0.1
snap grid on
vertices on
gizmo
```

Snapping afeta ferramentas editoriais; não quantiza silenciosamente expressões do console. **vertices on** exibe os oito cantos da caixa delimitadora. **gizmo** retorna diagnóstico da ferramenta.

16

Duplicação, matriz delta e repeat

Cópia simples

```
duplicate
duplicate count 10
```

Sem transformação, as cópias ocupam o mesmo lugar. A seleção passa para as cópias.

Sequência constante

```
duplicate count 12 move 1 0 0 rotate 0 15 0 scale 0.98 0.98 0.98
```

Quando todos os argumentos são numéricos, o runtime compõe uma matriz de passo e aplica potências sucessivas. Essa é a semântica clássica $M_k = \Delta^k M_0$.

Repeat

```
duplicate
move 1 0 0
rotate 0 15 0
repeat
```

A primeira transformação confirmada depois de `duplicate` define a matriz delta. `repeat` cria a próxima fronteira. Se a história ficar obsoleta, o sistema limpa a referência em vez de operar sobre IDs apagados.

Programa paramétrico

```
duplicate count 41 move 1 0 0 scale "0.2 + 0.8 * abs(sin(u * pi))" 1 1
```

A presença de expressão aciona o caminho paramétrico. O programa é compilado uma vez e avaliado para cada índice. O resultado informa `repeatSupported: false` no 0021d.

17

Gramática matemática

Operadores

Precedência aproximada	Operadores	Observação
maior	parênteses e chamadas	$\sin(x)$, $\max(a,b)$
potência	<code>**</code> , alias <code>^</code>	<code>^</code> normaliza para <code>**</code>
unário	<code>+</code> , <code>-</code>	sinal
produto	<code>*</code> , <code>/</code> , <code>%</code>	multiplicação, divisão, módulo
soma	<code>+</code> , <code>-</code>	adição e subtração

Acesso arbitrário como `objeto.propriedade` não pertence à gramática. Nomes e funções são resolvidos em listas permitidas, reduzindo a superfície de execução.

Constantes e variáveis

Nome	Significado
<code>i</code> , <code>index</code>	índice atual, começando em 1
<code>count</code>	número de cópias solicitadas
<code>u</code>	parâmetro normalizado $(i-1)/(count-1)$
<code>t</code> , <code>time</code>	tempo fornecido ao avaliador
<code>dt</code> , <code>deltaTime</code>	passo temporal
<code>x</code> , <code>y</code> , <code>z</code>	posição da transformação anterior
<code>sx</code> , <code>sy</code> , <code>sz</code>	escala anterior
<code>pi</code>	π
<code>tau</code>	2π
<code>e</code>	número de Euler
<code>phi</code>	razão áurea $(1+\sqrt{5})/2$
<code>deg</code> , <code>rad</code> , <code>turn</code>	conversores de unidade angular

Quando `count = 1`, `u = 0`. Para `count > 1`, a primeira cópia tem `u = 0` e a última `u = 1`.

Funções permitidas

```
sin cos tan · sind cosd tand · asin acos atan atan2
sqrt cbrt abs exp · log log10 · min max
floor ceil round trunc sign hypot
```

Trigonometria comum recebe radianos. As variantes terminadas em `d` recebem graus.

Unidades

```
30 deg  
pi/6 rad  
0.5 turn  
cosd(30)  
cos(pi/6)
```

Operações `rotate` do runtime recebem graus; `rad` converte radianos para graus e `turn` converte voltas para graus. Funções trigonométricas continuam matematicamente convencionais.

18

Semântica indexada

No modo `indexed`, o runtime conserva uma semente e uma transformação anterior. Para cada índice:

- 1 constrói o contexto `i`, `u`, `count`, tempo, posição e escala;
- 2 avalia as expressões;
- 3 soma `move` à posição anterior;
- 4 compõe `rotate` com a rotação anterior;
- 5 calcula `scale = scale_semente × fator(i)`.

Essa assimetria é intencional. Translação e rotação descrevem passos; escala descreve o valor no índice. Ela impede que `scale "0.9"` se torne 0.9^i quando o autor queria uma escala constante de 90% da semente.



Figura 5 — Uma curva absoluta $f(i)$ é reconstruída por diferenças $\Delta p(i) = f(i) - f(i-1)$.

Curva absoluta por deltas

Se desejamos centros em $p_i = f(i)$, mas `move` é incremental, usamos:

$$\Delta p_i = f(i) - f(i-1)$$

Então:

$$p_i = p_0 + \sum_{k=1}^i \Delta p_k = p_0 + f(i) - f(0).$$

Escolhendo $p_0 = f(0)$, obtemos $p_i = f(i)$. Hélice, roseta e Trindade Orbital usam essa identidade telescópica.

Modo recursive

A AST admite `recursive`, no qual cada passo é uma matriz composta sobre a anterior. O console 0021d não expõe uma palavra-chave para alternar o modo; sua superfície paramétrica padrão é `indexed`. O livro documenta `recursive` como opção semântica interna, não como comando disponível.

AST canônica e independência da sintaxe

A fonte é normalizada e compilada para uma AST versionada. Cada operação recebe ID semântico, por exemplo:

```
move    → transform.translation.delta  
rotate  → transform.rotation.delta  
scale   → transform.scale.factor-at
```

A AST registra:

- versão da AST;
- hash canônico;
- linguagem e versão de origem;
- modo `indexed` ou `recursive`;
- espaço de translação;
- operações e expressões normalizadas.

Palavras-chave em português, inglês ou uma interface gráfica podem futuramente gerar a mesma AST. A persistência deve favorecer a estrutura semântica; a frase original continua útil para autoria e depuração.

Backend matemático substituível

O parser não depende de `Math` em cada regra. Um backend fornece literais, operações, chamadas e conversão numérica. Isso abre espaço para precisão alternativa, simbólica ou GPU, desde que o contrato de avaliação seja preservado.

Cor, material e textura

Material compartilhado

No Inspector, **Cor-base** pertence ao material legado/aparência. Alterá-la pode afetar objetos que compartilham a mesma aparência, conforme a normalização e o internamento do runtime.

Cor própria da instância

- 1 Selecione exatamente um objeto.
- 2 Abra **Inspector**.
- 3 Marque **Usar cor própria**.
- 4 Escolha **Cor da instância**.
- 5 Pressione **Aplicar ao sandbox**.

Duplicações preservam `instanceState.color`. Mudar apenas essa cor não cria novo lote nem material. `undo`, `redo`, salvar e abrir devem preservar o atributo.

No 0028e, Inspector e console convergem no `PropertyRegistry`. Cores literais e procedurais usam o mesmo serviço:

```
property set appearance.color #33aaff
property set instance.color #ff8a3d
property batch instance.color "hsl(300 * u,0.8,0.55)" scope=renderables
```

O escopo `renderables` abre grupos em descendentes visuais; o grupo lógico permanece intacto. A expressão é compilada uma vez e o lote inteiro é validado antes de um único comando e uma única entrada de histórico.

Estratégia cromática para programas

Use uma semente por família cromática. Ajuste as três sementes no Inspector antes de executar; depois selecione por ID e duplique. A roseta e a cidade deste livro usam essa técnica. O código geométrico não precisa conter hexadecimal para preservar a paleta.

Textura

O Inspector aceita arquivo ou URL/Data URL, repetição, offset, rotação e wrapping (`repeat`, `mirror`, `clamp`). Texturas pertencem à aparência e devem ser serializáveis. Para livros e testes públicos, prefira assets locais ou Data URLs estáveis; URLs externas podem desaparecer ou bloquear CORS.

21

Projetos, apresentação, testes e diagnósticos

Salvar, abrir e novo

Salvar exporta um arquivo `.spatialseed/JSON`. Abrir valida schema, restaura cena, aparência, editor e configuração do renderer. Novo limpa a cena. O console canônico possui IDs `project.save`, `project.open`, `project.new` e `project.inspect`, embora a superfície textual direta para conteúdo de arquivo seja mediada pela interface.

Apresentação

- F ou botão de tela cheia: fullscreen real do navegador;
- Tab ou **Cena**: somente a cena;
- Esc: restaura interface; em fullscreen o primeiro Esc pode ser consumido pelo navegador;
- painéis têm maximizar/restaurar e podem permanecer abertos simultaneamente;
- a barra pode ser horizontal, vertical ou flutuante;
- posições, dimensões e perfil de atalhos persistem localmente.

Botões e teclado identificam as mesmas ações semânticas. `Ctrl/Cmd+Z` desfaz a cena quando o viewport está ativo, mas preserva o undo textual quando o foco está num editor.

Animação de preview

O runtime temporal do 0028e é efêmero: transforma a projeção, não o documento.

```
animate spin speed=45 axis=y
animate wave amplitude=1 frequency=0.5 phase=0.35 mode=objects
animate color "hsl(60 * t + 360 * u, 0.8, 0.55)" mode=objects
animate pause
animate resume
animate stop
```

`mode=selection` trata cada raiz como unidade rígida. `mode=objects` expande grupos em objetos renderizáveis e permite fase ou programa diferente por alvo. O painel também compõe faixas distintas. `animate stop` restaura exatamente matrizes e cores; nenhum frame entra no undo ou no arquivo `.spatialseed`.

Testes

```
test help
test all
runtime test help
runtime test affine-math
runtime test affine-repeat
runtime test instance-batches
runtime test property-contract
runtime test experiment-panel
runtime test ui-actions
runtime test animation-runtime
runtime test animation-tracks
runtime test all
```

As suítes verificam contratos em camadas. Um teste verde não prova estética ou FPS; prova o conjunto específico de invariantes codificados.

Recursos e benchmarks

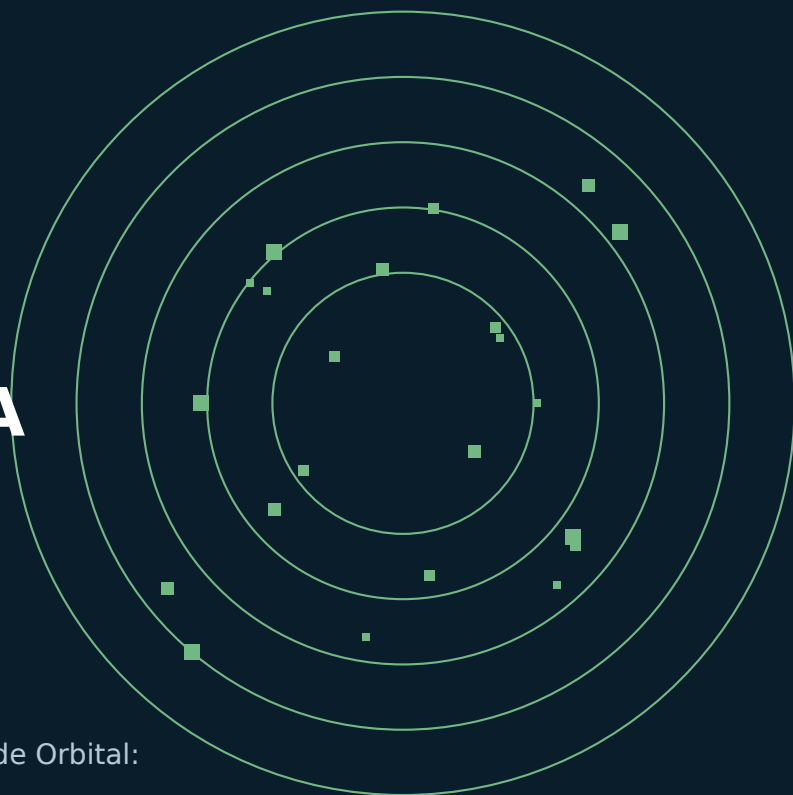
```
runtime resources
benchmark scene 1000 5 100
benchmark compare
benchmark history
runtime benchmark api 10000
```

`runtime resources` informa objetos, lotes, materiais, geometria e memória nominal. `benchmark scene` mede cenários no dispositivo. `runtime benchmark api` compara chamada direta e fachada. Resultados só são comparáveis quando dispositivo, versão, aquecimento e parâmetros são registrados.

ATLAS PROCEDURAL

GERAÇÃO PROCEDURAL: DA REGRA À CIDADE

Quatro novos programas graduais e a Trindade Orbital:
fórmulas, código, variações e validação.



Gerar proceduralmente não é pedir ao computador que “invente alguma coisa”. É escolher uma semente, um domínio de índices, uma família de transformações e os invariantes que devem permanecer visíveis.

22

O modelo mental

Todo exemplo desta parte pode ser lido em seis passos:

- 1 **semente:** objeto com identidade, geometria, aparência e transformação inicial;
- 2 **domínio:** quantidade de cópias e parâmetro normalizado;
- 3 **regra:** expressões de movimento, rotação, escala e pivô;
- 4 **avaliação:** contexto diferente para cada índice;
- 5 **materialização:** objetos lógicos e instâncias gráficas;
- 6 **projeção:** câmera, luz, seleção e modo de apresentação.

A mesma regra pode produzir um objeto didático com 20 amostras ou uma obra densa com 700. A diferença de contagem não deve alterar a ideia semântica.

Frequência, fase e amplitude

Três parâmetros reaparecem:

- **amplitude** controla o tamanho do desvio;
- **frequência** controla quantos ciclos cabem no domínio;
- **fase** desloca uma família sem mudar sua forma básica.

Em $A \cdot \sin(\omega u + \phi)$, A é amplitude, ω é frequência angular e ϕ é fase. Usar **tau** torna ciclos legíveis: $\sin(3 \cdot \text{tau} \cdot u)$ completa três ciclos quando u percorre $[0, 1]$.

Discreto e contínuo

O sistema produz cópias discretas; a fórmula pode descrever uma curva contínua. A quantidade controla amostragem. Poucas cópias revelam a construção; muitas aproximam um fio. A geometria de cada semente continua visível, portanto densidade também é decisão estética.

Custo

O custo lógico cresce aproximadamente com o número de objetos e operações avaliadas. O custo gráfico depende de lotes, materiais, transparência, helpers, seleção e dispositivo. Diminuir **count** é a primeira forma de adaptar uma obra a um celular; não é necessário simplificar a fórmula.

23

Exemplo 1: Onda Integrada

A primeira obra usa `move` incremental diretamente. Em vez de definir a altura absoluta, cada cópia acrescenta um seno à altura anterior:

$$y_i = y_{i-1} + 0,28 \cdot \sin(2\tau u_i).$$

A soma discreta do seno produz duas grandes inflexões e retorna aproximadamente ao nível inicial. A rotação constante faz cada cubo girar cinco graus em relação ao anterior; a escala pulsa três vezes.

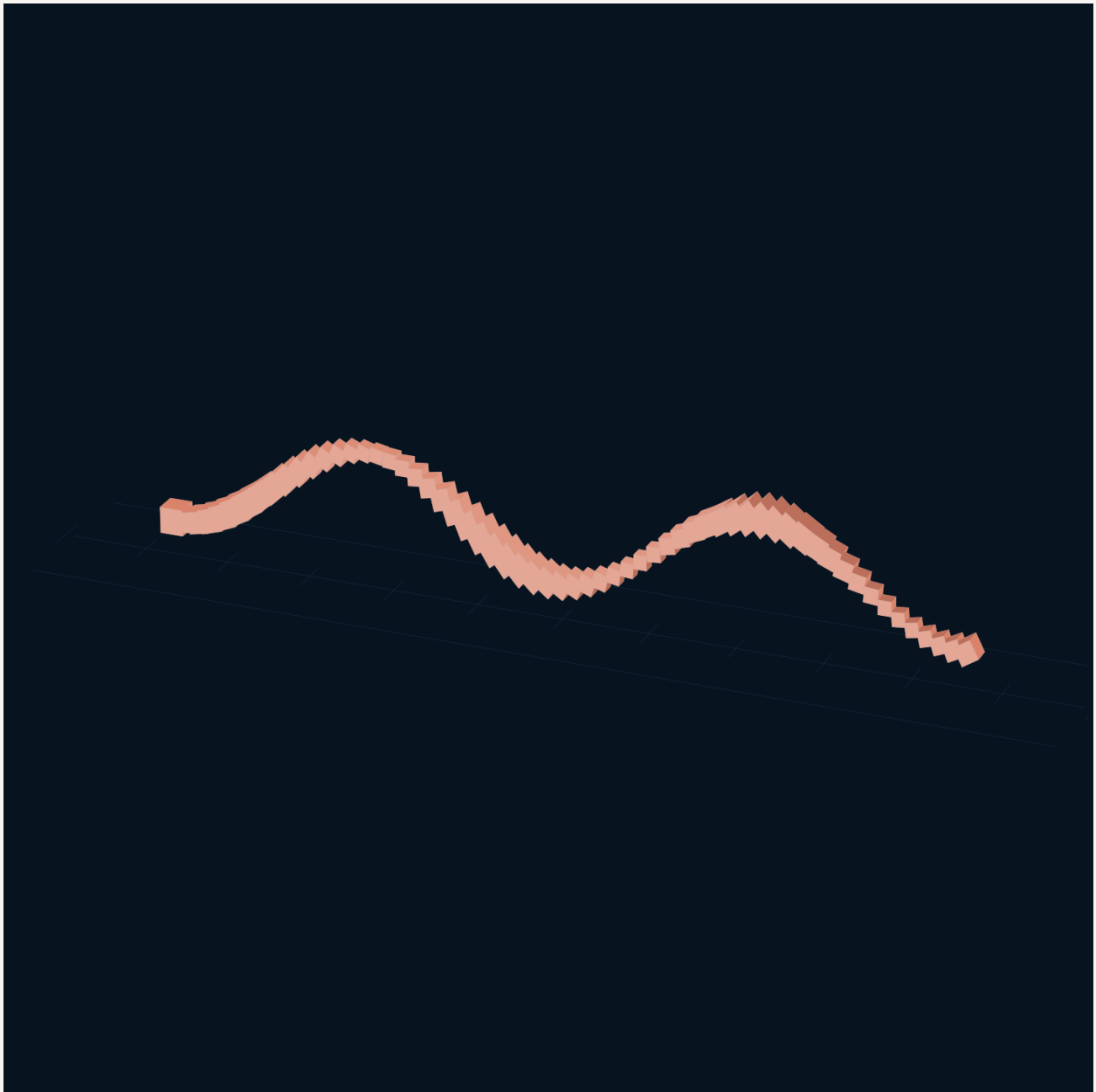


Figura 6 — Pré-visualização editorial calculada a partir das transformações validadas da Onda Integrada; não é captura do WebGL.

```

select only box-2
position -12 1 0
scale 0.35 0.35 0.35
duplicate count 60
  move 0.4 "0.28 * sin(2 * tau * u)" 0
  rotate 0 0 5
  scale "0.55 + 0.45 * (0.5 + 0.5 * sin(6 * pi * u))" ...
select clear

```

O bloco acima está quebrado para leitura. O arquivo executável `01_onda_integrada_0021d.txt` está anexado ao PDF e reproduzido integralmente no apêndice.

O que aprender

- `move` acumula;
- `rotate` acumula;
- `scale` é relativo à semente no índice;
- `u` percorre exatamente a extensão do programa;
- selecionar uma semente cromática basta para herdar sua cor.

Variações

- troque `2*tau*u` por `4*tau*u` para aumentar oscilações;
- reduza `0.4` para adensar os cubos em X;
- use `sin(2*tau*u+tau/4)` para deslocar a fase;
- substitua `rotate 0 0 5` por `rotate 3 5 7` para torção tridimensional.

Validação

O programa executou 5 comandos, produziu 63 objetos totais — 61 corais, uma semente azul e uma verde — e terminou com seleção vazia. A validação lógica no contêiner teve mediana de aproximadamente 4,8 ms em sete execuções; esse número não inclui renderer nem navegador.

24

Exemplo 2: Hélice Ascendente

Agora a posição desejada é absoluta:

$$f(i) = (4 \cos(\tau i/40), -4 + 0,05i, 4 \sin(\tau i/40)).$$

O programa fornece ao `move` a diferença $f(i) - f(i-1)$. Quarenta passos completam uma volta; 160 cópias completam quatro voltas e sobem oito unidades.

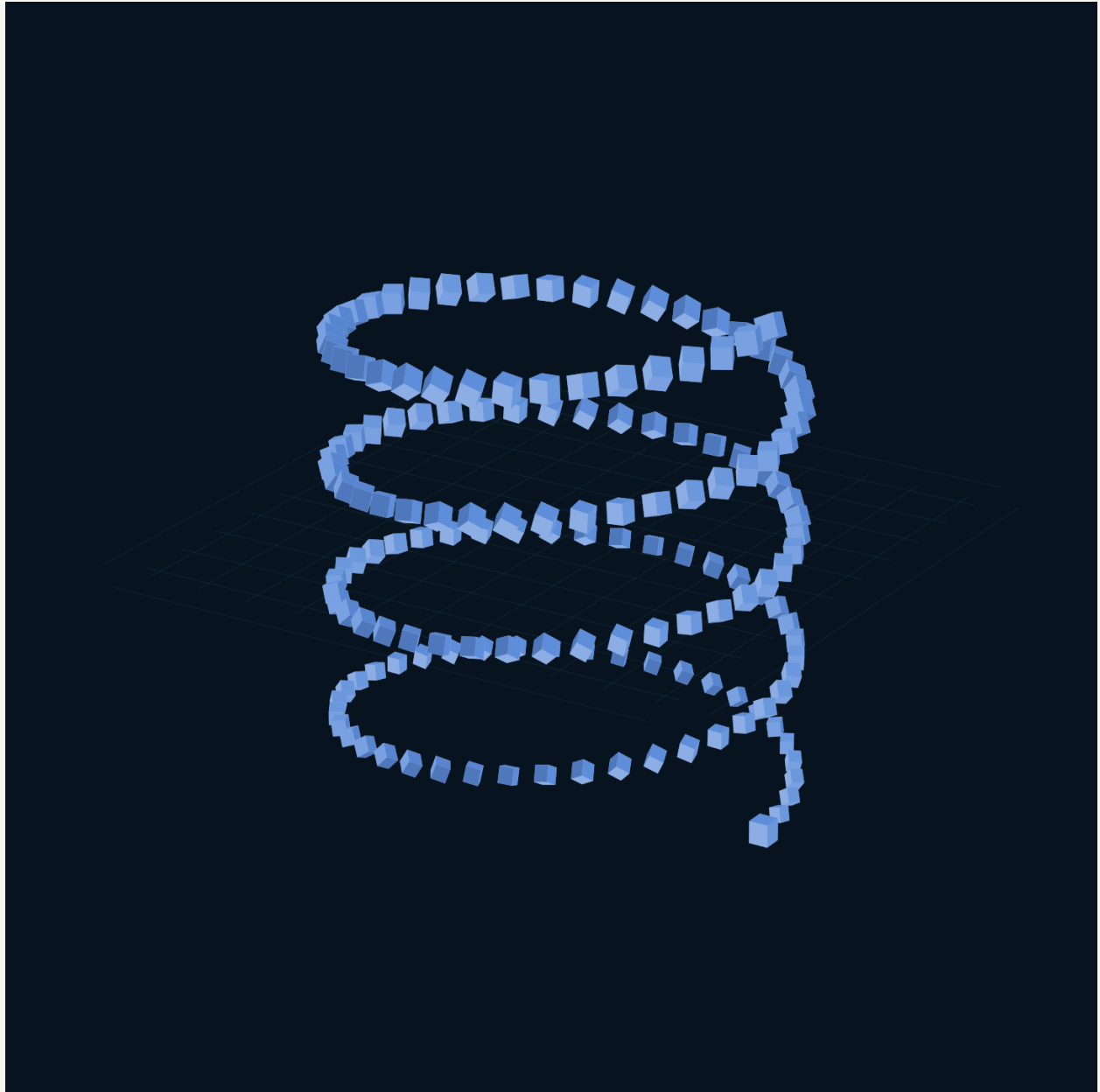


Figura 7 — Pré-visualização da Hélice Ascendente: quatro voltas, crescimento gradual e rotação tridimensional.


```

select only box-1
position 4 -4 0
scale 0.18 0.18 0.18
duplicate count 160 move
  "4 * cos(tau * i / 40) - 4 * cos(tau * (i - 1) / 40)"
  0.05
  "4 * sin(tau * i / 40) - 4 * sin(tau * (i - 1) / 40)"
  rotate 7 11 13
  scale "0.65 + 0.35 * u" "0.65 + 0.35 * u" "0.65 + 0.35 * u"
select clear

```

Por que a fórmula fecha em XZ

No índice 160, $\tau \cdot 160 / 40 = 4\tau$; seno e cosseno retornam ao ponto inicial. A altura não fecha porque recebe incremento constante. A obra combina periodicidade horizontal e deriva vertical.

Variações

- $\tau \cdot i / 20$ produz oito voltas com a mesma contagem;
- troque 0.05 por "0.02+0.04*u" para acelerar a subida;
- use raios diferentes em X e Z para uma hélice elíptica;
- aplique fases diferentes às três sementes para uma hélice tripla.

Validação

Foram criados 163 objetos totais, dos quais 161 azuis. Os centros ficaram entre -4 e 4 em X e Z e entre -4 e 4 em Y. A mediana lógica foi aproximadamente 5,0 ms; novamente, não é medição gráfica.

25

Exemplo 3: Roseta Tricromática

A roseta usa três sementes e uma curva polar:

$$r(t, \varphi) = 4 \cos(3t + \varphi)$$

$$x = r \cos t$$

$$z = r \sin t$$

$$y = 0,5 \sin(6t + \varphi).$$

As fases 0 , $\tau/3$ e $2\tau/3$ relacionam três famílias sem sobreposição completamente. A pequena oscilação vertical separa cruzamentos e torna a roseta tridimensional em vista oblíqua.

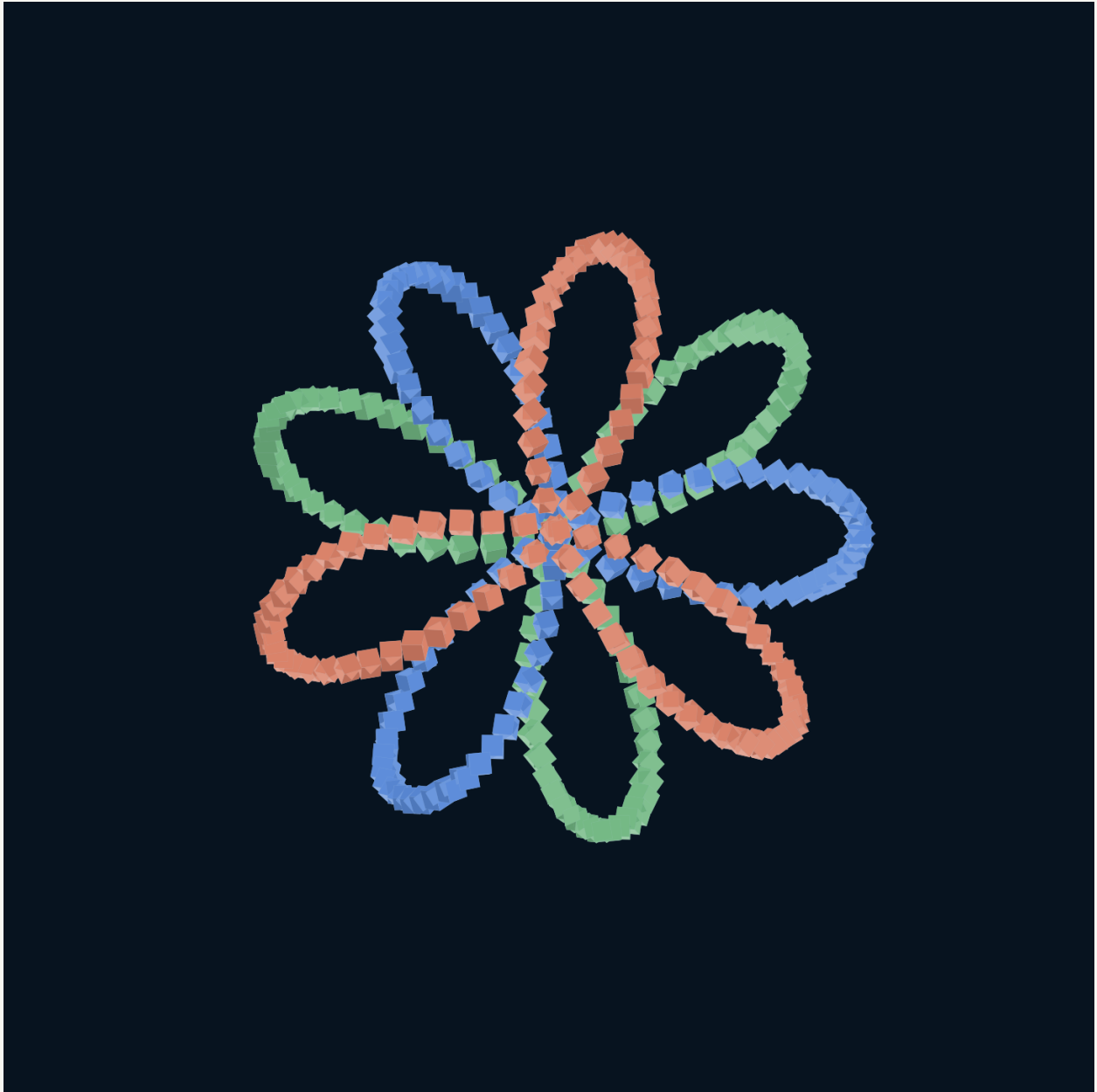


Figura 8 — Pré-visualização editorial da Roseta Tricromática. Cada cor é uma semente e uma fase do mesmo programa.

Código de uma fase

```
select only box-1
position 4 0 0
scale 0.14 0.14 0.14
duplicate count 180 move
  "4*cos(3*tau*i/count)*cos(tau*i/count)
  -4*cos(3*tau*(i-1)/count)*cos(tau*(i-1)/count)"
  "0.5*sin(6*tau*i/count)
  -0.5*sin(6*tau*(i-1)/count)"
  "4*cos(3*tau*i/count)*sin(tau*i/count)
  -4*cos(3*tau*(i-1)/count)*sin(tau*(i-1)/count)"
rotate 5 7 11
scale "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u))" ...
```

As outras duas fases acrescentam $\tau/3$ e $2\tau/3$ a cada argumento de fase e selecionam `box-2` e `box-3`.

Cor como parâmetro externo

O programa não contém os hexadecimais. As cores pertencem às sementes. Antes de executar, o autor pode abrir o Inspector e substituir a paleta, por exemplo:

Família	Paleta diurna	Paleta noturna
box-1	#5b8bd9	#5ce1e6
box-2	#d98067	#ff4d8d
box-3	#72b883	#9dff6c

Duplicar depois da mudança preserva a cor própria/material da semente. Essa separação permite reutilizar a mesma geometria como identidade cromática diferente.

Validação

O programa completo executou 13 comandos, produziu 543 objetos — 181 por cor — e terminou com seleção vazia. O erro de fechamento de cada fase ficou no nível de arredondamento de ponto flutuante. A geração lógica teve mediana aproximada de 31,7 ms no contêiner.

26

Exemplo 4: Cidade Policêntrica

A cidade demonstra que geração procedural não precisa produzir apenas curvas. Ela combina:

- dois distritos de 4×8 edifícios;
- alturas calculadas por funções diferentes;
- bases alinhadas ao plano $y = 0$;
- rotações incrementais opostas;
- quinze eixos verdes formados por paralelepípedos alongados;
- três famílias cromáticas, cada uma herdada de uma semente.

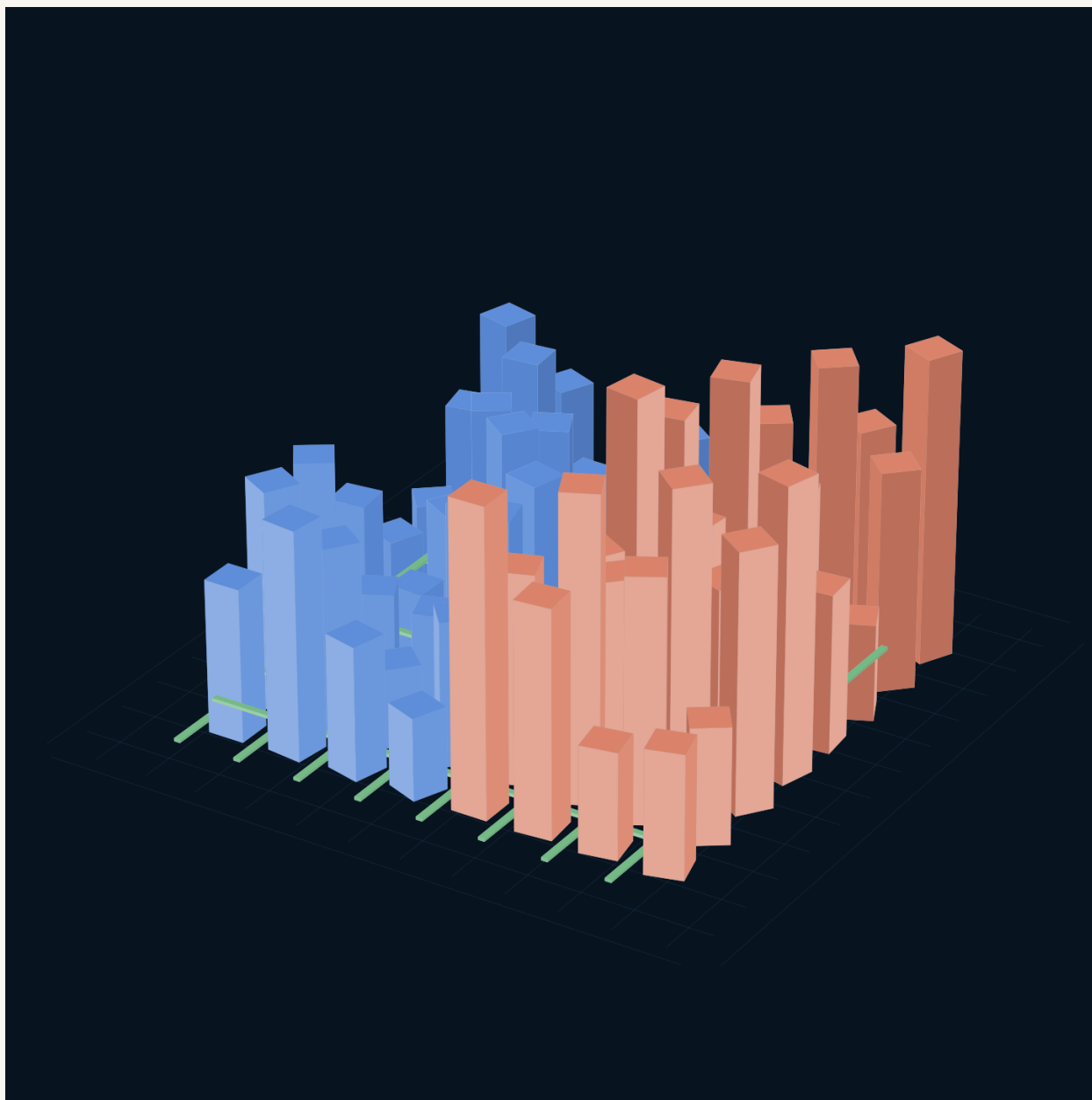


Figura 9 — Pré-visualização editorial da Cidade Policêntrica, calculada dos 79 objetos validados. A cena real pode ser reproduzida no GitHub Pages.

Índice bidimensional dentro de um índice linear

Para uma malha com quatro colunas:

$$\text{col}(i) = i \% 4$$

$$\text{row}(i) = \text{floor}(i / 4).$$

A posição absoluta é $x = x_0 + 2,4 \cdot \text{col}(i)$ e $z = z_0 + 2,4 \cdot \text{row}(i)$. Como `move` é incremental, o comando usa diferenças entre índice atual e anterior:

```
move
"2.4 * ((i % 4) - ((i - 1) % 4))"
Δy(i)
"2.4 * (floor(i / 4) - floor((i - 1) / 4))"
```

Quando a coluna volta de 3 para 0, o delta X recua 7,2 unidades e o delta Z avança uma rua. O cursor percorre a malha em varredura.

Altura com base no chão

No distrito azul:

$$H(i) = 2,5 + 5 \cdot [0,5 + 0,5 \sin(1,7i + 0,55 \text{ floor}(i/4))].$$

O objeto tem tamanho-base 2. Para obter altura física H , a escala Y deve ser $H/2$ e o centro deve estar em $H/2$. A semente usa $H(0)=5$, portanto começa com centro $y=2,5$ e escala $sy=2,5$. Para cada cópia:

```
move_y(i) = [H(i)-H(i-1)]/2
scale_y(i) = H(i)/H(0).
```

Assim a escala paramétrica produz $sy = H(i)/2$ e a soma dos deltas posiciona o centro em $H(i)/2$. A base permanece em zero.

Eixos verdes

```
select only box-3
position 0 0.05 -8.4
scale 9 0.05 0.12
duplicate count 7 move 0 0 2.4

select only box-3
position -8.4 0.05 0
rotate 0 90 0
duplicate count 7 move 2.4 0 0
```

A primeira família cria linhas paralelas em Z; a segunda gira a semente e cria linhas em X. As cópias anteriores preservam sua orientação.

Distrito azul — forma explicada

```
select only box-1
position -7.2 2.5 -7.2
scale 0.68 2.5 0.68
duplicate count 31
  move Δx(i) "(H(i) - H(i - 1)) / 2" Δz(i)
  rotate 0 7 0
  scale 1 "H(i) / 5" 1
```

O arquivo executável expande $H(i)$ integralmente. O distrito coral repete a estrutura com cosseno, amplitude maior, fase espacial distinta e rotação -5° por índice.

Possibilidades urbanas

- altere o módulo de 4 para 6 e ajuste `count` para bairros mais largos;
- use frequências baixas para zonas de altura suave;

- use `abs(sin(...))` para evitar vales simétricos negativos;
- escolha pivô absoluto e rotação para distritos radiais;
- duplique uma semente achatada para praças, plataformas ou lâminas d'água;
- use texturas no Inspector antes da duplicação para fachadas compartilhadas;
- salve diferentes paletas como projetos sem alterar o programa geométrico.

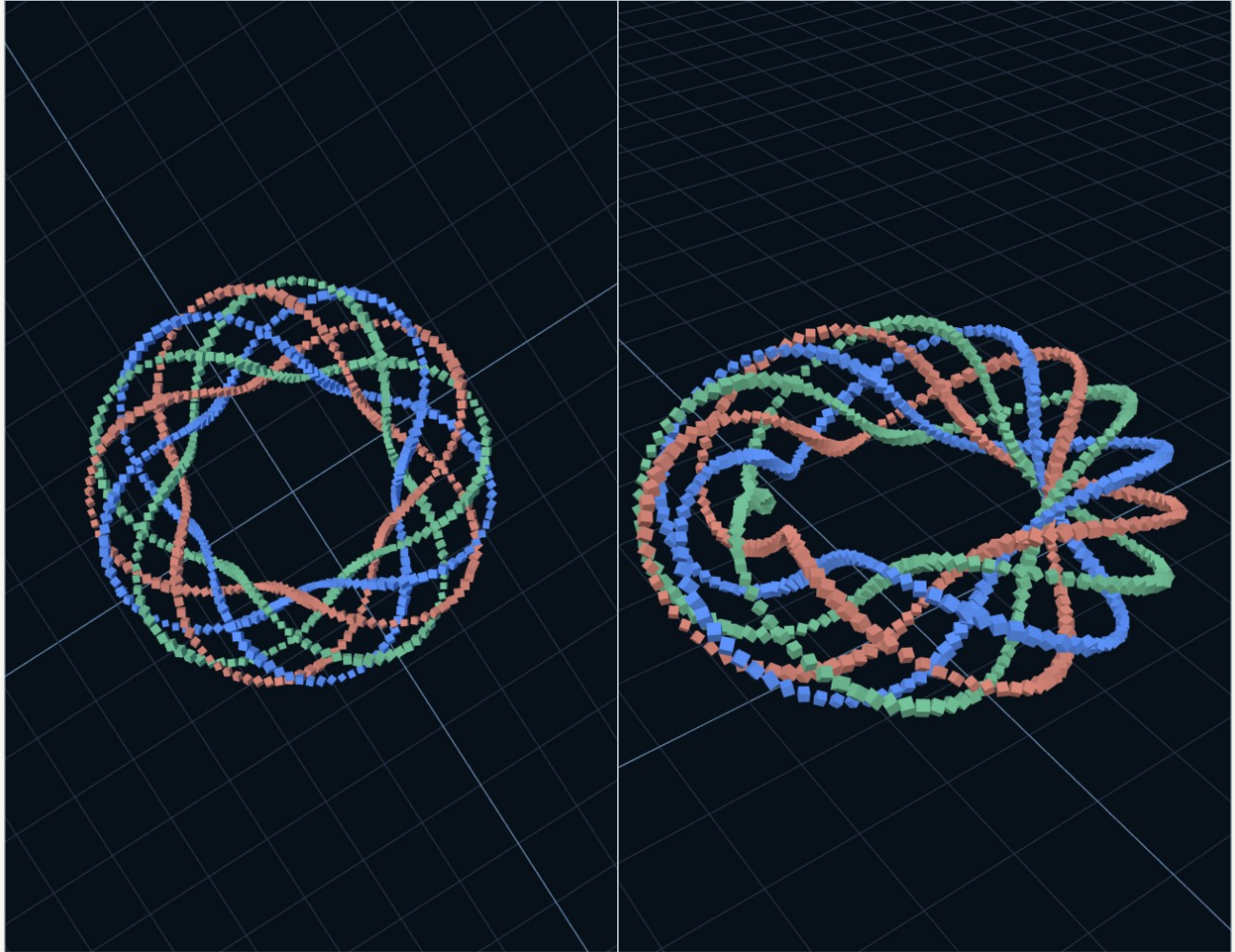
Validação

O programa executou 17 comandos e gerou 79 objetos: 32 edifícios azuis, 32 corais e 15 eixos verdes. A seleção final ficou vazia. A mediana lógica foi aproximadamente 3,3 ms. Essa rapidez reflete a pequena contagem e o ambiente sem renderer; a experiência móvel deve ser medida separadamente.

27

Obra avançada: Trindade Orbital

A Trindade Orbital combina três fios sobre uma curva toroidal modulada. Cada fio usa 240 cópias e uma fase cromática. Com as sementes, a cena contém 723 objetos — 241 por cor.



Vista superior real, executada no SpatialSeed público. O enquadramento remove margens vazias e preserva a coroa.

Vista oblíqua real no navegador móvel; a barra superior confirma o GitHub Pages.

Estrutura

Se $t = \tau i / \text{count}$, raio maior $R=4$, raio menor $r=1,25$ e fase φ , a curva é:

$$\rho(t, \varphi) = R + r \cos(5t + \varphi)$$

$$x = \rho \cos(2t)$$

$$y = r \sin(5t + \varphi)$$

$$z = \rho \sin(2t).$$

As frequências 2 e 5 são coprimas; a trajetória percorre a estrutura antes de fechar. As fases 0 , $\tau/3$ e $2\tau/3$ geram três fios relacionados.

Pulsção e orientação

A escala usa doze ciclos:

$$s(u, \varphi) = 0,72 + 0,28 \cdot [0,5 + 0,5 \sin(12\tau u + \varphi)].$$

A rotação incremental 7° , 11° , 13° evita que as caixas conservem uma orientação plana. Como os ângulos são diferentes e relativamente desencontrados, as faces produzem variação ao longo do fio.

Evidência e autoria

O programa foi concebido e validado por OpenAI Codex em diálogo com Rogério Duarte. Uma restrição do navegador remoto impediu a abertura do domínio publicado; Rogério executou no Android, escolheu câmera e devolveu as capturas. O resultado é conjunto e rastreável. Não é imagem sintetizada por modelo: é geometria calculada e renderizada pelo SpatialSeed. As menções a OpenAI, GitHub e Microsoft descrevem ferramentas e infraestrutura, não patrocínio ou endosso institucional.

Validação

Foram processados 13 comandos sem erro. A contagem foi 723; cada cor teve 241 objetos; a seleção final ficou vazia. O erro de fechamento dos três fios ficou aproximadamente entre 8×10^{-15} e $1,3 \times 10^{-14}$ unidades.

Código integral — Trindade Orbital

Listagem integral. As setas ↵ indicam apenas quebra visual da linha; para executar, use o arquivo textual anexado ao PDF.

01	select only box-1
02	position 5.25 3.5 0
03	scale 0.12 0.12 0.12
04	<pre>duplicate count 240 move "(4 + 1.25 * cos(5 * tau * i / count + 0)) * cos(2 * tau * i / count) ↳ - (4 + 1.25 * cos(5 * tau * (i - 1) / count + 0)) * cos(2 * tau * (i - 1) / count)" ↳ "1.25 * sin(5 * tau * i / count + 0) - 1.25 * sin(5 * tau * (i - 1) / count + 0)" "(4 + ↳ 1.25 * cos(5 * tau * i / count + 0)) * sin(2 * tau * i / count) - (4 + 1.25 * cos(5 * ↳ tau * (i - 1) / count + 0)) * sin(2 * tau * (i - 1) / count)" rotate 7 11 13 scale "0.72 ↳ + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + 0))" "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * ↳ u + 0))" "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + 0))"</pre>
05	select only box-2
06	position 3.375 4.5825317547 0
07	scale 0.12 0.12 0.12
08	<pre>duplicate count 240 move "(4 + 1.25 * cos(5 * tau * i / count + tau / 3)) * cos(2 * tau * i / ↳ count) - (4 + 1.25 * cos(5 * tau * (i - 1) / count + tau / 3)) * cos(2 * tau * (i - 1) / ↳ count)" "1.25 * sin(5 * tau * i / count + tau / 3) - 1.25 * sin(5 * tau * (i - 1) / ↳ count + tau / 3)" "(4 + 1.25 * cos(5 * tau * i / count + tau / 3)) * sin(2 * tau * i / ↳ count) - (4 + 1.25 * cos(5 * tau * (i - 1) / count + tau / 3)) * sin(2 * tau * (i - 1) / ↳ count)" rotate 7 11 13 scale "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + tau / 3))" ↳ "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + tau / 3))" "0.72 + 0.28 * (0.5 + 0.5 * ↳ sin(12 * tau * u + tau / 3))"</pre>
09	select only box-3
10	position 3.375 2.4174682453 0

11	scale 0.12 0.12 0.12
12	<pre>duplicate count 240 move "(4 + 1.25 * cos(5 * tau * i / count + 2 * tau / 3)) * cos(2 * tau * ↳ i / count) - (4 + 1.25 * cos(5 * tau * (i - 1) / count + 2 * tau / 3)) * cos(2 * tau * ↳ (i - 1) / count)" "1.25 * sin(5 * tau * i / count + 2 * tau / 3) - 1.25 * sin(5 * tau * ↳ (i - 1) / count + 2 * tau / 3)" "(4 + 1.25 * cos(5 * tau * i / count + 2 * tau / 3)) * ↳ sin(2 * tau * i / count) - (4 + 1.25 * cos(5 * tau * (i - 1) / count + 2 * tau / 3)) * ↳ sin(2 * tau * (i - 1) / count)" rotate 7 11 13 scale "0.72 + 0.28 * (0.5 + 0.5 * sin(12 ↳ * tau * u + 2 * tau / 3))" "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + 2 * tau / 3))" ↳ "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + 2 * tau / 3))"</pre>
13	select clear

Laboratório de variações

A listagem integral é também um mapa de parâmetros. Para conservar o fechamento da obra, altere sempre o termo no índice atual *i* e o correspondente no índice anterior *i-1*. Se apenas um lado mudar, a diferença discreta deixa de telescopar e surge uma costura. Todo o procedimento usa capacidades presentes no 0021d — seleção por ID, posição, escala, duplicação parametrizada, *move*, *rotate* e trigonometria — sem gerador externo oculto.

Parâmetro	Onde aparece	Efeito visual	Regra de segurança
count = 240	quantidade de duplicações	resolução e densidade do fio	pode mudar livremente; o denominador <i>count</i> se ajusta
raio maior 4	$4 + 1.25 * \cos(\dots)$	tamanho geral da coroa	mantenha maior que o raio menor para evitar atravessamento central
raio menor 1.25	posição e oscilação vertical	espessura do toro	altere todas as ocorrências, atuais e anteriores
frequências 2 e 5	ângulo maior e onda menor	família do nó toroidal	inteiros coprimos produzem um único fio fechado
frequência 12	expressão de escala	número de pulsações	inteiro positivo preserva o encontro das extremidades
fases 0, $\tau/3$, $2*\tau/3$	segundo e terceiro fios	separação e entrelaçamento cromático	use o mesmo deslocamento em posição e escala
rotações 7 11 13	operação <i>rotate</i>	torção das caixas	valores pequenos dão continuidade; valores altos tornam o fio mais áspero

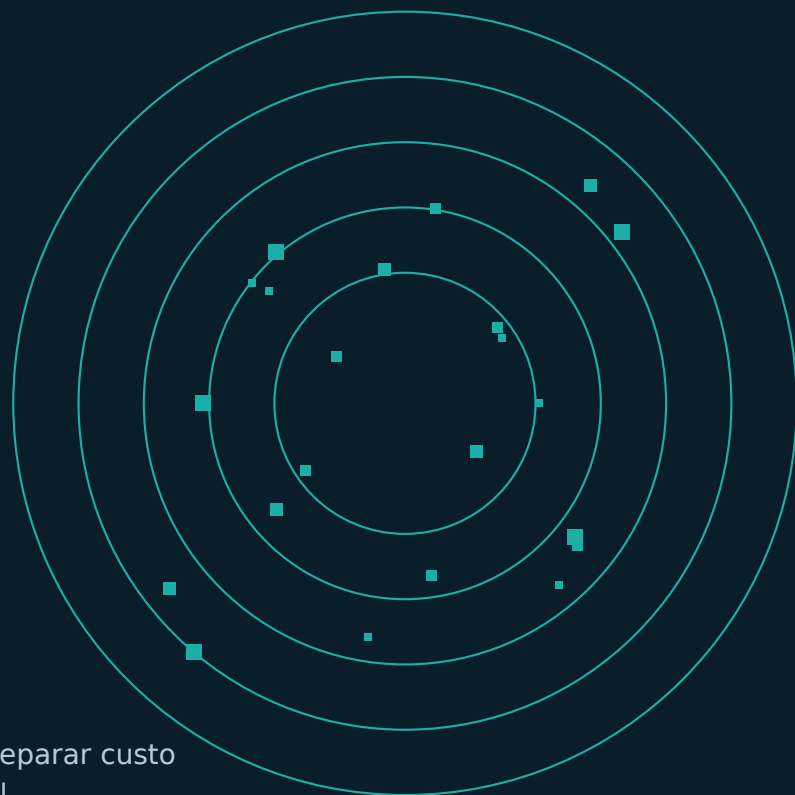
Uma variação “trevo” pode trocar a frequência menor 5 por 3; uma trama mais densa pode usar o par coprimo 3 e 7. A paleta não está embutida no comando: selecione cada semente no Inspector, atribua a cor desejada e só então execute o bloco correspondente. Assim, geometria e aparência continuam desacopladas.

Reprodução no sistema público

- 1 Abra um projeto novo e confirme a existência de *box-1*, *box-2* e *box-3*.
- 2 Ajuste a cor-base ou a cor própria das três sementes no Inspector.
- 3 Abra o Console e execute cada linha do arquivo anexado na ordem apresentada.
- 4 Aguarde a conclusão de uma linha antes de enviar a seguinte; no celular, isso torna falhas localizáveis. Ao terminar, use *select clear*, feche os painéis e enquadre a câmera em vista superior ou oblíqua.

EVIDÊNCIA

PERFORMANCE, TESTES E HONESTIDADE EXPERIMENTAL



O que foi medido, o que falta medir e como separar custo lógico, fachada, recursos e experiência móvel.

Performance não é um adjetivo. É uma relação entre versão, dispositivo, cena, aquecimento, métrica e pergunta.

28

O que foi medido

Fachada de runtime

Em Android/Termux, em 14 de julho de 2026, três execuções de 10.000 chamadas `noop` compararam acesso direto ao `CommandRegistry` e acesso pela fachada `SpatialSeedRuntime`.

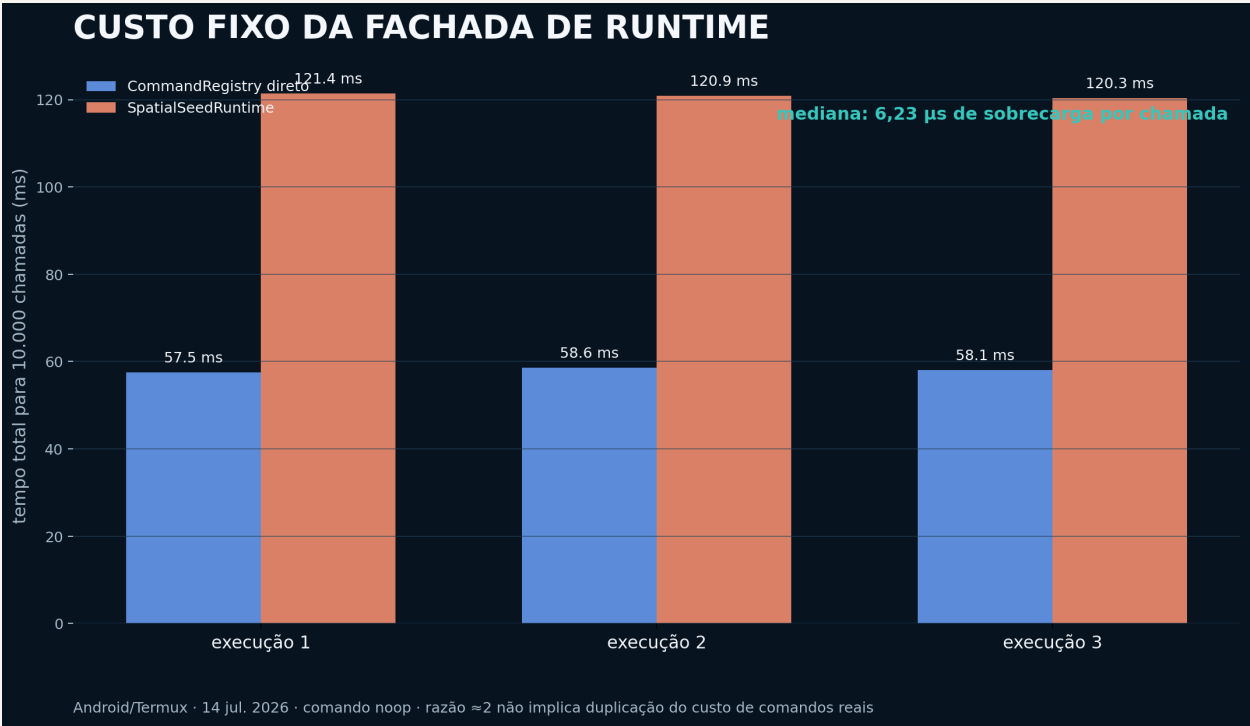


Figura 12 — Custo fixo da fachada de runtime em três execuções Android/Termux.

Execução	Direto	Fachada	Sobrecarga	Por chamada	Razão
1	57,5 ms	121,4 ms	63,9 ms	6,39 μs	2,111
2	58,6 ms	120,9 ms	62,3 ms	6,23 μs	2,063
3	58,1 ms	120,3 ms	62,2 ms	6,22 μs	2,071

A mediana foi **6,23 μs por chamada**. A razão próxima de dois mede um `noop`, em que o custo fixo domina. Não significa que um comando geométrico real dobra de preço.

Avaliação afim

Versão	Teste	Transformações	Total	Por transformação	Contexto
0020b-b	paramétrico	1.000	19,5 ms	0,0195 ms	melhor execução observada
0020b-b	paramétrico	1.000	36,4 ms	0,0364 ms	após reinicialização
0020b-b	nativo	10.000	197,5 ms	0,01975 ms	Android/Termux
0020b-c	paramétrico	1.000	23,1 ms	0,0231 ms	AffineProgram extraído
0020b-c	nativo	10.000	193,1 ms	0,01931 ms	após extração

A variação 19,5–36,4 ms para o mesmo tamanho mostra por que uma única execução não é benchmark. Aquecimento, coleta de lixo e carga do dispositivo precisam ser controlados.

Cor por instância

O contrato registra **12 bytes por capacidade de instância** e espera zero novos materiais, geometrias, lotes ou draw calls ao mudar somente a cor. O CSV de 0020b-f deixou o campo temporal “PREENCHER”. Esta edição não inventa um número ausente.

Cena padrão após a fachada

O relatório registrou 3 objetos lógicos, 3 lotes instanciados, 1 geometria única, 3 materiais e 17 chamadas de renderização no instante medido. Draw calls incluem elementos auxiliares e dependem do estado visual.

29

O que os novos exemplos medem — e o que não medem

Os quatro programas da v0.5 foram executados sete vezes contra parser, AST, seleção, sandbox e gerador afim em Node no contêiner. As medianas foram:

Programa	Objetos totais	Comandos	Mediana lógica
Onda Integrada	63	5	~4,8 ms
Hélice Ascendente	163	5	~5,0 ms
Roseta Tricromática	543	13	~31,7 ms
Cidade Policêntrica	79	17	~3,3 ms

Esses números incluem tokenização, execução do console e geração lógica. Não incluem:

- DOM e layout dos painéis;
- criação/atualização WebGL;
- compilação de shaders;
- câmera e interação;
- composição da tela pelo navegador;
- temperatura e bateria do celular.

Portanto não devem ser comparados diretamente com os números Android/Termux anteriores. Eles funcionam como regressão local e ordem de grandeza, não como alegação de experiência do usuário.

30

Como medir corretamente

Protocolo mínimo

- 1 Registre commit, tag e build visível.
- 2 Registre aparelho, navegador, sistema e estado térmico.
- 3 Recarregue e espere a cena estabilizar.
- 4 Execute uma rodada de aquecimento que não entra no resultado.
- 5 Faça pelo menos cinco amostras.
- 6 Registre mediana, mínimo, máximo e parâmetros.
- 7 Separe tempo lógico, tempo de atualização do renderer e FPS.
- 8 Registre objetos, lotes, materiais, geometria e draw calls.
- 9 Preserve o programa exato junto do resultado.

Comandos úteis

```
runtime resources
runtime benchmark api 10000
benchmark scene 1000 7 100
benchmark history
runtime test all
```

Teste visual

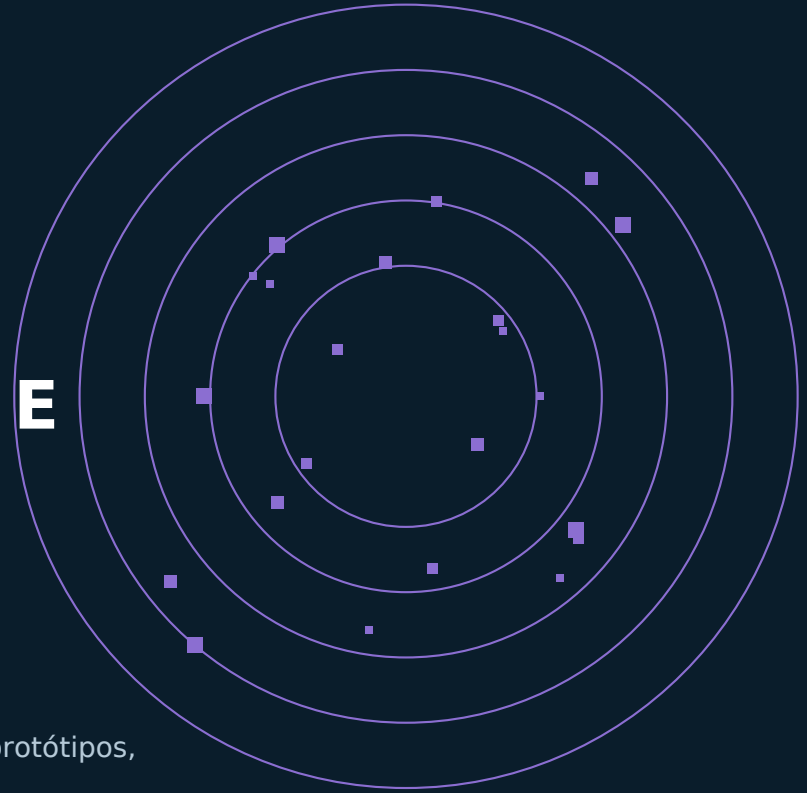
Depois das suítes, verifique:

- seleção temporária restaura a cor lógica;
- undo/redo preserva cor, transformação e identidade;
- salvar/abrir preserva a cena;
- pivô e gizmo concordam com o console;
- modo cena não altera o mundo;
- programas consecutivos não deixam seleção obsoleta;
- curvas fechadas retornam à semente dentro do erro numérico esperado.

HORIZONTE

POSSIBILIDADES E ROTEIRO

Assets procedurais, parâmetros, aparência, protótipos,
regiões, diffs semânticos e múltiplos viewers.



SpatialSeed é útil já como laboratório de linguagem espacial. Sua arquitetura, porém, foi escolhida para permitir crescimento sem transformar cada novidade em exceção.

31

Possibilidades imediatas

Biblioteca procedural

Procedimentos e experimentos já preservam fonte e parâmetros fora da cena. O passo seguinte é tratá-los como assets com miniatura, versão, dependências, licença e vínculo explícito ao projeto sem executar código durante a importação.

Parâmetros nomeados

O laboratório de experimentos já oferece parâmetros nomeados e controles gerados. A evolução é levar o mesmo contrato a procedimentos e animações persistentes, sem criar schemas incompatíveis para cada superfície.

Comandos de aparência

`property set`, `property unset` e `property batch` já levam aparência, textura, transformação e cor de instância ao mesmo contrato do Inspector. O trabalho restante é ampliar providers sem perder atomicidade, valores mistos e compartilhamento de recursos.

Geração bidimensional e urbana

Funções `row`, `col` e domínios multidimensionais poderiam tornar grids mais legíveis. A cidade deste livro prova que `%` e `floor` bastam, mas também revela o valor de abstrações nomeadas.

Grupos e protótipos

Grupos aninháveis e transformações locais já existem. Ainda falta persistir receitas procedurais compactas, promover famílias a protótipos editáveis e definir copy-on-write para regenerar estruturas sem destruir exceções locais.

Segundo viewer

O outline já oferece uma segunda projeção diagnóstica. Um viewer funcionalmente independente, textual, documental ou WebGPU ampliaria o teste de substituíbilidade. O critério não seria pixel idêntico, mas preservação de IDs, transforms, aparência lógica, seleção e deltas observáveis.

Possibilidades distribuídas

Regiões autoritativas

Cada região pode publicar schema, capacidades e política de aceitação. Um sandbox local carrega versão-base e envia `CommitEnvelope`. A autoridade retorna delta, rejeição ou necessidade de rebase.

Diff semântico

Comparar JSON bruto confunde ordem, valores derivados e identidades. Um diff semântico poderia dizer: “12 instâncias do protótipo P foram transformadas pela AST H” em vez de listar centenas de números.

Rebase de programas

Se a região mudou desde a versão-base, reaplicar um programa requer verificar dependências: a semente ainda existe? O protótipo mudou? A política de pivô continua válida? A AST e seu hash tornam essa pergunta tratável.

Mundos federados

Uma região não precisa aceitar todas as regras de outra. Integração difere de invasão. Um protocolo federado deve permitir descobrir capacidades, converter representações e manter fronteiras de autoridade.

Motivações para continuar

O projeto reúne três linhas que normalmente ficam separadas:

- **criação visual**, porque a linguagem precisa produzir formas que valham a pena habitar;
- **semântica verificável**, porque imagem sem contrato não sustenta continuidade;
- **autoria distribuída**, porque pessoas, agentes e módulos podem contribuir sem dissolver responsabilidade.

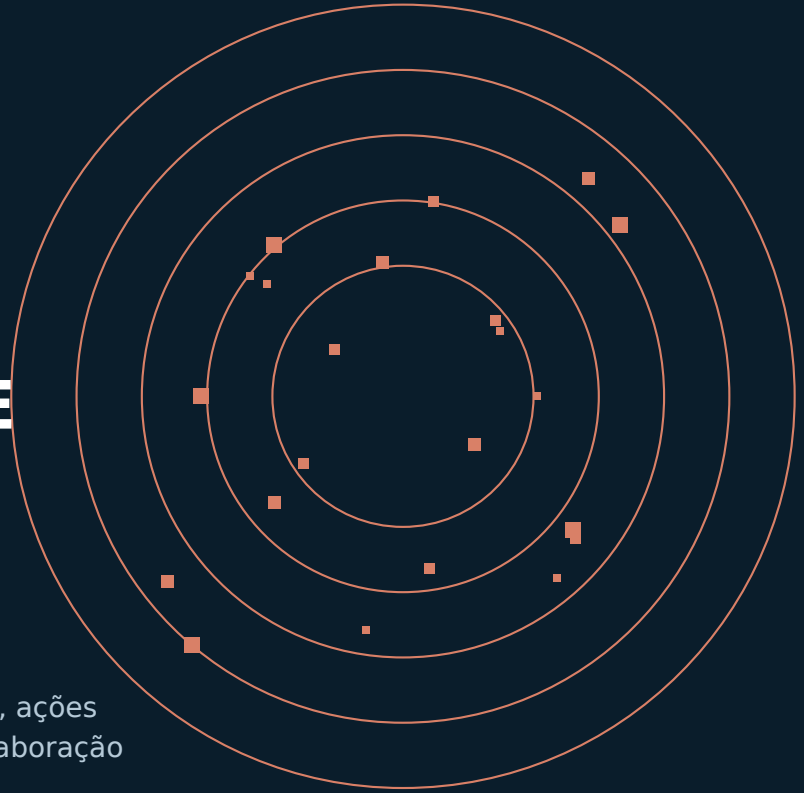
A cidade e a Trindade Orbital mostram dois extremos. Uma organiza ocupação e infraestrutura; a outra explora simetria e fase. Ambas nascem da mesma pequena linguagem. Essa continuidade entre arte, arquitetura e protocolo é a razão mais forte para aprofundar o SpatialSeed.

O sistema não precisa prever todas as formas. Precisa preservar a relação que permite que novas formas apareçam sem perder identidade, autoria e caminho de volta.

CONTINUIDADE

TEMPO, EXPERIMENTOS E COLABORAÇÃO

Laboratórios declarativos, animação efêmera, ações configuráveis e um método verificável de colaboração com LLMs.



A evolução entre 0021d e 0028e transformou vários itens da antiga seção de possibilidades em capacidades reais. Esta parte registra essa mudança sem reescrever retrospectivamente a história dos exemplos.

34

Do 0022 ao 0028e

Ciclo	Resultado implementado	Fronteira preservada
0022	propriedades tipadas, Inspector e console em lote	<code>PropertyRegistry</code> é a fonte comum
0023	grupos aninháveis e transformações locais	hierarquia lógica independe de Three.js
0024	famílias geométricas e interface configurável	providers e manifesto evitam listas paralelas
0025	PWA e transporte de arquivos	offline não finge salvar a cena
0026	Worker/SES, planos e procedimentos	programas planejam antes de publicar
0027	laboratório declarativo e otimizações de autoria	experimentos não recebem DOM nem sandbox
0028	tempo efêmero, ações, lotes e animação	preview não se torna estado editorial

Esse percurso mostra uma regra de crescimento: primeiro definir a fronteira, depois ligar mais superfícies a ela. O Inspector não ganhou um interpretador privado; passou a emitir o mesmo comando de propriedades. O painel de experimentos não ganhou acesso direto à cena; passou a gerar o mesmo plano do console. A animação não escreveu matrizes no sandbox a cada frame; ganhou uma sobreposição restaurável no renderer.

Evidência do marco

No 0028d, o autor executou `runtime test all` no navegador e confirmou 337 testes aprovados, nenhuma falha. Depois, no 0028e, confirmou visualmente que a alça central branca escala X, Y e Z proporcionalmente. Esses números pertencem à evidência desta edição, não a uma promessa permanente: o runtime atual deve ser sempre consultado.

Laboratórios declarativos

Um experimento é uma definição serializável com identidade, texto, parâmetros e programa. O host conhece um conjunto fechado de widgets; o experimento escolhe entre eles, mas não fornece HTML, CSS ou handlers.

```
experiment list
experiment show math.helix
experiment helix radius=4 turns=5 count=160 color=#5b8bd9
plan status
plan commit
```

A sequência separa duas ações. `experiment ...` calcula e devolve um plano; `plan commit` publica esse plano depois da revisão. Alterar um slider apenas muda estado local do painel até que o usuário peça nova geração.

Contrato de equivalência

Para a mesma definição, parâmetros normalizados, semente, snapshot e revisão, painel e console devem produzir o mesmo plano. A equivalência é testável porque ambos usam `ExperimentService` e `ProgramSessionController`, não duas versões da geometria.

Limites

O plugin é interno. Ele não instala código externo, não publica uma API geral de plugins e não prova segurança para JavaScript hostil. Esses limites são deliberados: uma interface declarativa segura não deve ser usada como pretexto para conceder capacidades que o contrato ainda não auditou.

Tempo sem corromper a autoria

Uma animação contínua produz muitos frames, mas nem todo frame é uma decisão editorial. O 0028e separa o relógio do documento: o runtime avalia programas, aplica matrizes e cores efêmeras e pode restaurar a projeção original.

As variáveis principais são:

- **t**: tempo do runtime, em segundos;
- **i**: índice inteiro da unidade animada;
- **u**: índice normalizado entre zero e um;
- **count**: quantidade total de unidades.

Exemplos:

```
animate orbit radius=4 speed=30 axis=y
animate rainbow speed=60 saturation=0.8 mode=objects
animate move "2 * sin(t)" 0 0
animate rotate 0 "90 * t + 20 * sin(tau * t)" 0
```

No modo **selection**, cada raiz é capturada como unidade. Um grupo gira rigidamente em torno de sua âncora. No modo **objects**, descendentes renderizáveis recebem índices próprios; assim uma única seleção pode produzir fases, trajetórias e cores diferentes.

Faixas

O painel de animação permite compor várias faixas. Cada faixa conserva alvos, preset e parâmetros. Antes de reproduzir, o serviço resolve IDs, detecta sobreposição ambígua e compila programas. Todas as faixas compartilham o mesmo relógio e overlay.

Cor temporal

animate color usa **hsl**, **rgb** ou **mix**. A cor temporária vai ao atributo de instância, não a um material novo por frame. Parar remove a sobreposição e reapresenta a cor canônica. Isso preserva batching e evita transformar aparência visual em mutação silenciosa.

O que ainda falta decidir

Persistir uma animação exige identidade de clip, canais, duração, vínculos de alvo, keyframes ou expressões, serialização e regras de conflito. Eventos, colisões e scripts anexados também precisam de orçamento e autoridade. O overlay atual prova o mecanismo temporal; não antecipa essas decisões no formato do projeto.

37

Colaboração verificável com LLMs

Uma LLM pode acelerar pesquisa, implementação, teste e redação, mas sua saída é uma proposta. O repositório, os testes e a observação no aparelho continuam sendo fontes de evidência. O método adotado procura conservar aprendizado, autoria e reversibilidade.

Hierarquia documental

- 1 AGENTS.md: entrada operacional para assistentes;
- 2 PROJECT_SEED.md: semente curta de retomada;
- 3 docs/project/CHATGPT_PROJECT_INSTRUCTIONS.md: contrato detalhado;
- 4 docs/project/WORKFLOW.md: branches, patches, autoria e integração;
- 5 docs/project/DECISIONS.md: decisões duráveis;
- 6 código, testes, manifesto de build e ajuda do runtime: estado executável.

Se a memória de uma conversa contradiz o branch carregado, prevalece o branch. Se um documento vivo contradiz código e testes, o documento deve ser corrigido.

Ciclo de trabalho

- 1 confirmar diretório, branch, HEAD, status, remoto e build;
- 2 localizar os módulos, contratos e testes envolvidos;
- 3 declarar objetivo, fronteira e critério de aceite;
- 4 implementar um incremento pequeno;
- 5 executar validação estática, suíte específica e regressão ampla;
- 6 testar visualmente no Android quando houver interação;
- 7 produzir patch auditável;
- 8 o autor aplica, testa, publica e integra quando decidir;
- 9 atualizar documentação e ponto de retomada.

Patch canônico

O fluxo preferido transporta um commit por `git format-patch` e aplica com `git am`. Uma entrega reproduzível informa:

```
EXPECTED_BASE=<pai exato>
EXPECTED_SHA=<sha256 do patch>
EXPECTED_COMMIT=<commit após aplicação>
```

Árvore suja, base divergente ou SHA incorreto são condições de parada. Forçar o patch destruiria justamente a evidência que os hashes oferecem.

Autoria

A autoria principal permanece:

```
Rogério Duarte <rd@rogerioduarte.org>
```

A assistência técnica pode ser registrada separadamente:

```
Assisted-by: OpenAI Codex
```

Gerar texto ou código não transfere autoria, direção nem decisão. O objetivo da colaboração é aumentar a capacidade do autor e deixar um caminho que ele possa auditar, reproduzir e continuar.

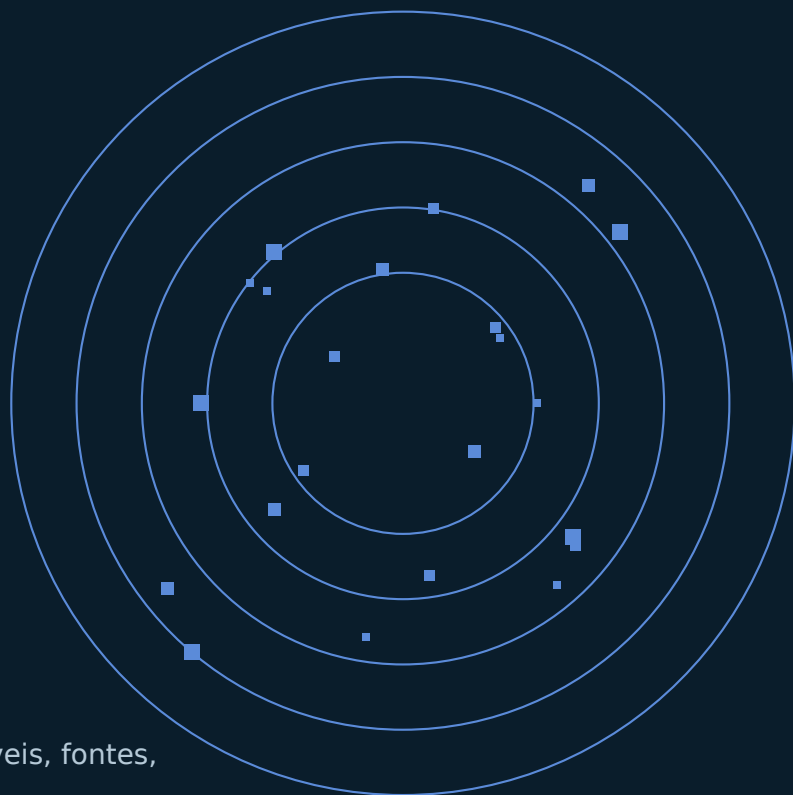
Regra de honestidade

Documentação deve distinguir implementado, testado, decisão, requisito, opção e horizonte. Uma suíte verde não apaga uma regressão visual. Uma demonstração não estabelece segurança de produção. Uma possibilidade arquitetural não deve ser vendida como recurso presente. Essa disciplina é o equivalente documental da separação entre preview e commit.

REFERÊNCIA

APÊNDICES

Comandos, matemática, programas executáveis, fontes, limites e checklist de reprodução.



Os apêndices concentram consulta rápida e os programas completos. Os arquivos .txt também estão incorporados ao próprio PDF como anexos.

APÊNDICE A

Referência rápida de comandos

Comando	Uso
<code>help</code>	ajuda resumida
<code>commands</code>	capacidades registradas
<code>inspect selection</code>	snapshot da seleção
<code>inspect selected [all]</code>	objeto ativo ou todos selecionados
<code>inspect input/editor/sandbox/region/objects</code>	diagnósticos por camada
<code>list objects</code>	objetos do sandbox
<code>select only/add/remove/toggle ID...</code>	edição explícita da seleção
<code>select clear, clear</code>	limpar seleção
<code>create box [x y z]</code>	criar caixa
<code>position x y z</code>	posicionar pivô da seleção
<code>move dx dy dz</code>	transladar seleção
<code>rotate x y z</code>	girar em graus
<code>scale x y z</code>	escalar seleção
<code>pivot median/bounds/active</code>	política de pivô
<code>pivot absolute x y z</code>	pivô mundial
<code>pivot relative dx dy dz</code>	pivô relativo ao ativo
<code>duplicate</code>	uma cópia
<code>duplicate count N</code>	N cópias
<code>duplicate count N op...</code>	sequência afim constante ou paramétrica
<code>repeat</code>	reaplicar matriz delta constante
<code>group nome, ungroup</code>	criar ou abrir hierarquia
<code>delete</code>	excluir seleção
<code>undo, redo</code>	histórico local
<code>property list/inspect</code>	descobrir e consultar propriedades
<code>property set/unset</code>	editar seleção pelo registro comum
<code>property batch id "expressão"</code>	lote procedural atômico
<code>procedure define/run/import/export</code>	catálogo de procedimentos
<code>experiment list/show/id</code>	laboratório declarativo e plano
<code>plan status/commit/discard</code>	revisar, aplicar ou abandonar plano
<code>animate preset ...</code>	iniciar animação efêmera

Comando	Uso
<code>animate</code> <code>pause/resume/stop/status</code>	controlar e restaurar preview
<code>vertices on/off</code>	marcadores de bounds
<code>snap move/rotate/scale valor</code>	snapping
<code>snap grid on/off</code>	grid lock
<code>gizmo</code>	diagnóstico do gizmo
<code>test help/all/suite</code>	testes básicos
<code>runtime test help/suite/all</code>	testes de camadas
<code>runtime resources</code>	auditoria de recursos
<code>runtime benchmark api N</code>	custo da fachada
<code>benchmark scene [objetos]</code> <code>[amostras] [transformados]</code>	benchmark de cena
<code>benchmark compare/history/clear</code>	histórico de benchmarks

APÊNDICE B

Referência matemática

Grupo	Símbolos
Variáveis	i index count u t time dt deltaTime x y z sx sy sz
Constantes	pi tau e phi deg rad turn
Trigonometria	sin cos tan sind cosd tand asin acos atan atan2
Raízes e expoentes	sqrt cbrt exp log log10
Discretização	floor ceil round trunc sign
Agregação	min max hypot abs
Operadores	+ - * / % **, alias ^

APÊNDICE C

Programas executáveis

C.1 — Onda Integrada

Arquivo 01_onda_integrada_0021d.txt

Listagem integral. As setas ↵ indicam apenas quebra visual da linha; para executar, use o arquivo textual anexado ao PDF.

01	select only box-2
02	position -12 1 0
03	scale 0.35 0.35 0.35
04	duplicate count 60 move 0.4 "0.28 * sin(2 * tau * u)" 0 rotate 0 0 5 scale "0.55 + 0.45 * (0.5 ↳ + 0.5 * sin(6 * pi * u))" "0.55 + 0.45 * (0.5 + 0.5 * sin(6 * pi * u))" "0.55 + 0.45 * ↳ (0.5 + 0.5 * sin(6 * pi * u))"
05	select clear

C.2 — Hélice Ascendente

Arquivo 02_helice_ascendente_0021d.txt

Listagem integral. As setas ↵ indicam apenas quebra visual da linha; para executar, use o arquivo textual anexado ao PDF.

01	select only box-1
02	position 4 -4 0
03	scale 0.18 0.18 0.18
04	duplicate count 160 move "4 * cos(tau * i / 40) - 4 * cos(tau * (i - 1) / 40)" 0.05 "4 * ↳ sin(tau * i / 40) - 4 * sin(tau * (i - 1) / 40)" rotate 7 11 13 scale "0.65 + 0.35 * u" ↳ "0.65 + 0.35 * u" "0.65 + 0.35 * u"
05	select clear

C.3 — Roseta Tricromática

Arquivo 03_roseta_tricromatica_0021d.txt

Listagem integral. As setas ↵ indicam apenas quebra visual da linha; para executar, use o arquivo textual anexado ao PDF.

01	select only box-1
02	position 4 0 0
03	scale 0.14 0.14 0.14
04	duplicate count 180 move "4 * cos(3 * tau * i / count + 0) * cos(tau * i / count) - 4 * cos(3 ↳ * tau * (i - 1) / count + 0) * cos(tau * (i - 1) / count)" "0.5 * sin(6 * tau * i / ↳ count + 0) - 0.5 * sin(6 * tau * (i - 1) / count + 0)" "4 * cos(3 * tau * i / count + 0) ↳ * sin(tau * i / count) - 4 * cos(3 * tau * (i - 1) / count + 0) * sin(tau * (i - 1) / ↳ count)" rotate 5 7 11 scale "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + 0))" "0.65 + ↳ 0.35 * (0.5 + 0.5 * sin(9 * tau * u + 0))" "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + ↳ 0))"
05	select only box-2

06	position -2 0.4330127019 0
07	scale 0.14 0.14 0.14
08	duplicate count 180 move "4 * cos(3 * tau * i / count + tau / 3) * cos(tau * i / count) - 4 * ↳ cos(3 * tau * (i - 1) / count + tau / 3) * cos(tau * (i - 1) / count)" "0.5 * sin(6 * ↳ tau * i / count + tau / 3) - 0.5 * sin(6 * tau * (i - 1) / count + tau / 3)" "4 * cos(3 ↳ * tau * i / count + tau / 3) * sin(tau * i / count) - 4 * cos(3 * tau * (i - 1) / count ↳ + tau / 3) * sin(tau * (i - 1) / count)" rotate 5 7 11 scale "0.65 + 0.35 * (0.5 + 0.5 * ↳ sin(9 * tau * u + tau / 3))" "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + tau / 3))" ↳ "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + tau / 3))"
09	select only box-3
10	position -2 -0.4330127019 0
11	scale 0.14 0.14 0.14
12	duplicate count 180 move "4 * cos(3 * tau * i / count + 2 * tau / 3) * cos(tau * i / count) - ↳ 4 * cos(3 * tau * (i - 1) / count + 2 * tau / 3) * cos(tau * (i - 1) / count)" "0.5 * ↳ sin(6 * tau * i / count + 2 * tau / 3) - 0.5 * sin(6 * tau * (i - 1) / count + 2 * tau / ↳ 3)" "4 * cos(3 * tau * i / count + 2 * tau / 3) * sin(tau * i / count) - 4 * cos(3 * tau ↳ * (i - 1) / count + 2 * tau / 3) * sin(tau * (i - 1) / count)" rotate 5 7 11 scale "0.65 ↳ + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + 2 * tau / 3))" "0.65 + 0.35 * (0.5 + 0.5 * sin(9 ↳ * tau * u + 2 * tau / 3))" "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + 2 * tau / 3))"
13	select clear

C.4 — Cidade Policêntrica

Arquivo 04_cidade_policentrica_0021d.txt

Listagem integral. As setas ↳ indicam apenas quebra visual da linha; para executar, use o arquivo textual anexado ao PDF.

01	select only box-3
02	position 0 0.05 -8.4
03	scale 9 0.05 0.12
04	duplicate count 7 move 0 0 2.4
05	select only box-3
06	position -8.4 0.05 0
07	rotate 0 90 0
08	duplicate count 7 move 2.4 0 0
09	select only box-1
10	position -7.2 2.5 -7.2
11	scale 0.68 2.5 0.68
12	duplicate count 31 move "2.4 * ((i % 4) - ((i - 1) % 4))" "((2.5 + 5 * (0.5 + 0.5 * sin(1.7 * ↳ i + 0.55 * floor(i / 4)))) - (2.5 + 5 * (0.5 + 0.5 * sin(1.7 * (i - 1) + 0.55 * floor((i ↳ - 1) / 4)))) / 2" "2.4 * (floor(i / 4) - floor((i - 1) / 4))" rotate 0 7 0 scale 1 ↳ "(2.5 + 5 * (0.5 + 0.5 * sin(1.7 * i + 0.55 * floor(i / 4)))) / 5" 1
13	select only box-2
14	position 2.4 5 -7.2
15	scale 0.68 5 0.68

```

16 duplicate count 31 move "2.4 * ((i % 4) - ((i - 1) % 4))" "((3 + 7 * (0.5 + 0.5 * cos(1.31 * i
    ↳ - 0.43 * floor(i / 4)))) - (3 + 7 * (0.5 + 0.5 * cos(1.31 * (i - 1) - 0.43 * floor((i -
    ↳ 1) / 4)))) / 2" "2.4 * (floor(i / 4) - floor((i - 1) / 4))" rotate 0 -5 0 scale 1 "(3 +
    ↳ 7 * (0.5 + 0.5 * cos(1.31 * i - 0.43 * floor(i / 4)))) / 10" 1

17 select clear

```

C.5 — Trindade Orbital

O programa está apresentado no capítulo 27 e anexado como `SpatialSeed_Trindade_Orbital_0021d.txt`.

Como usar os anexos do PDF

Esta edição carrega seis anexos: os cinco programas textuais e `VALIDACAO_0021d.json`. Em leitores que exibem anexos, abra o painel identificado por um clipe e salve o arquivo desejado. Alguns visualizadores de navegador escondem esse painel; nesse caso, salve o PDF e abra-o em um leitor compatível.

Em sistemas com Poppler, a extração também pode ser feita no terminal:

```

pdfdetach -list SpatialSeed_Livro_Manual_e_Atlas_Procedural_v0.6.pdf
pdfdetach -saveall SpatialSeed_Livro_Manual_e_Atlas_Procedural_v0.6.pdf

```

Cada `.txt` contém uma instrução por linha. Execute-as de cima para baixo no Console. As setas ↳ impressas nas tabelas deste livro são somente marcas de quebra editorial e não pertencem à linguagem; os anexos preservam as linhas originais sem essas marcas.

O manifesto JSON registra, para cada exemplo, a quantidade de comandos, objetos, distribuição cromática, seleção final e mediana da geração lógica. Ele serve para conferir a reprodução, não como promessa de taxa de quadros: não inclui WebGL, DOM, câmera nem custo do dispositivo móvel.

APÊNDICE D

Evidência, fontes e limites

Fonte	Uso nesta edição
main, commit b4043c6, build 20260720-0028e	implementação autoritativa desta edição
docs/AFFINE_AST_AND_SEMANTICS_0021D.md	AST, modos e IDs semânticos
docs/AFFINE_MATH_SYNTAX_0020B_D.md	gramática, unidades e backend
docs/INSTANCE_COLOR_0020B_F.md	cor por instância e memória
docs/EXPERIMENT_PLUGIN_0027A.md	laboratório declarativo e fronteiras
docs/INTERACTION_SURFACE_0028C.md	ações, atalhos e foco
docs/ANIMATION_WORKSPACE_0028D.md	lotes, faixas e animação efêmera
AGENTS.md e docs/project/*	método de colaboração, decisões e roadmap
docs/DISTRIBUTED_WORLD_ARCHITECTURE.md	arquitetura-alvo e fronteiras
docs/performance/*	números temporais e lacunas declaradas
Capturas Android fornecidas por Rogério Duarte	evidência visual do viewer real
Validadores da edição v0.5	evidência histórica dos exemplos 0021d
Teste integral 337/337 e validação da escala uniforme	evidência relatada pelo autor para 0028d-e

As pré-visualizações da onda, hélice, roseta e cidade foram calculadas a partir dos objetos produzidos pelo runtime e renderizadas editorialmente fora do WebGL para permitir diagramação estável. Elas são identificadas como pré-visualizações. As capturas da interface e da Trindade Orbital são imagens reais do sistema em navegador móvel.

APÊNDICE E

Checklist de reprodução

- 1 Confirmar URL e build.
- 2 Salvar trabalho existente.
- 3 Recarregar as três sementes.
- 4 Ajustar cores no Inspector, se desejado.
- 5 Colar o arquivo `.txt` integral, sem comentários.
- 6 Executar uma vez.
- 7 Verificar saída sem erros.
- 8 Executar `runtime resources` se o objetivo for medir.
- 9 Limpar seleção.
- 10 Ativar Cena, enquadrar e capturar.
- 11 Registrar dispositivo, horário, contagem e commit.

Nota editorial da versão 0.6

Esta edição preserva o atlas procedural e a evidência da v0.5, mas move a referência técnica do 0021d para o `main` no 0028e. O manual agora inclui hierarquia, PWA, procedimentos, experimentos, ações configuráveis, lotes procedurais e animação efêmera. Uma parte própria registra o método de colaboração com LLMs, autoria, patches canônicos e a distinção entre implementado, testado, decisão, requisito, opção e horizonte.

O livro foi produzido em colaboração entre Rogério Duarte e OpenAI Codex. O GitHub Pages serve a aplicação pública; GitHub/Microsoft fornecem infraestrutura de distribuição. Essas relações não implicam patrocínio ou endosso institucional.